

UNIVERSITATEA „LUCIAN BLAGA” DIN SIBIU

FACULTATEA DE ȘTIINȚE

Specializarea: Informatică

LUCRARE DE LICENȚĂ

Coordonator științific

Lector univ. Dr. Maria Flori

Absolvent

Mihai Albu

Sibiu

anul 2025-2026

UNIVERSITATEA „LUCIAN BLAGA” DIN SIBIU

FACULTATEA DE ȘTIINȚE

Specializarea: Informatică

Proiectarea și implementarea unei mâini robotice controlate printr-o aplicație web în timp real

Coordonator științific

Lector univ. Dr. Maria Flori

Absolvent

Mihai Albu

Sibiu

anul 2025-2026

Cuprins

Introducere	1
Capitolul 1. Fundamente teoretice privind sistemele embedded și IoT	3
1.1 Arhitectura sistemelor embedded	3
1.2. Internet of Things (IoT).....	3
Capitolul 2. Prezentarea platformei ESP32	5
2.1. Arhitectura ESP32.....	5
2.2. Programarea in C++	6
Capitolul 3. Servere web asincrone în sisteme embedded	9
3.1. Modelul sincron și asincron	9
3.2. Evenimente și callback-uri.....	10
3.3. Gestionarea cererilor HTTP	10
Capitolul 4. Proiectarea sistemului mâinii robotice	12
4.1. Cerințe funcționale	12
4.2. Cerințe nefuncționale.....	13
4.2.1. Timpul de răspuns.....	13
4.2.2. Stabilitatea comunicației	14
4.3. Arhitectura generală a sistemului.....	14
Capitolul 5. Implementare hardware	17
5.1. Generalizare și aplicabilitatea industrială.....	17
5.2. Integrarea componentelor și construcția prototipului.....	18
Capitolul 6. Implementare software	21
6.1. Structura generală a aplicației.....	21
6.2. Implementarea serverului web asincron	23
6.3. Controlul servomotoarelor	25
6.4. Comunicarea între frontend și backend.....	28
6.5. Structura HTML, stilizare CSS și funcționalitate JavaScript	30
Capitolul 7. Testare și validare	33
7.1. Testare funcțională	34
7.2. Testare performanță, stres, rezultate.....	35
Concluzii.....	40
Bibliografie	43
Anexe.....	44
Anexa 1 - Codul sursă al modulului hand_gestures.cpp & web_server_connect.cpp	44
Anexa 2 - Codul sursă al modulului main.cpp & main_cfg.hpp.....	50

Introducere

Măinile robotice au fost destul de intens studiate în cercetare. O varietate de mâini a fost dezvoltată cu scopul de a îmbunătăți eficiența, precizia și funcționalitatea asemănătoare cu cea a unei mâini umane. Multe din aceste mâini propun o inovație diferită una de cealaltă, de la design până la funcționalitatea software, specifică pentru o anumită aplicație.

Printre aceste inovații se numără mâna acționată prin aliaje cu memorie de formă (SMA) echipată cu senzori de flexiune [5], mâinile cu mecanisme acționate prin tendoane și legături mecanice [6], dar și mâna multisenzorială DLR, care are un schelet deschis, proiectat pentru o mai bună mentenanță [7].

În ultimii ani, mâinile robotice s-au dezvoltat rapid datorită ariei largi de aplicații, precum operațiile de tip pick-and-place. În mod specific, mâinile robotice sunt utilizate într-un număr mare de aplicații, inclusiv în domeniul medical [8], unde aceste progrese ale mâinilor robotice redau o oarecare funcționalitate persoanelor cu dizabilități. În mediul industrial, mâinile robotice reduc dependența forței de muncă, făcând astfel ca eficiența și rentabilitatea să crească [9]. De asemenea, putem remarca faptul că mâinile robotice sunt utilizate și în explorarea spațială [10], fapt ce evidențiază contribuția semnificativă la avansarea cercetării și experimentării în medii extraterestre.

Evoluția mâinilor robotice a condus la o diversitate de arhitecturi capabile să reproducă diferite aspecte ale funcționalității unei mâini umane normale. Majoritatea proiectelor folosesc o acționare pe motoare, oferind o libertate ridicată de mișcare. Aceste motoare sunt alese deoarece au o precizie destul de mare, sunt ușor de controlat și au o greutate redusă.

Totuși, designurile mai simple de mâini robotice nu oferă precizia necesară pentru operații de înaltă acuratețe, comparabile cu cele ale unei mâini umane. Dacă se dorește depășirea acestor limitări, este nevoie de dezvoltarea unor proiecte moderat complexe, capabile să concureze la nivel de performanță și versatilitate cu o mână umană.

Prin faptul că această tematică este atât de abordată și de studiată, multe persoane au încercat să își creeze propria lor mână robotică, fie ea mai complexă sau mai simplă. Cu toate acestea, am realizat că multe persoane se concentrau pe partea hardware, dar neglijau partea software, care este la fel de importantă sau poate chiar mai importantă.

Această dorință de a crea o mână robotică, a ajutat la realizarea faptului că această tematică poate fi abordată și de către un student.

Aplicația dezvoltată constă într-un sistem integrat hardware-software pentru controlul unei mâini robotice cu cinci degete, acționate individual prin servomotoare, utilizând o interfață web accesibilă prin rețea. Sistemul este construit în jurul unui microcontroler ESP32. Acest microcontroler a fost ales datorită comunicației Wi-Fi integrate, dar și datorită puterii de procesare ridicate.

Mâna robotică este proiectată astfel încât fiecare deget să fie controlat independent prin intermediul unui servomotor, rezultând un total de cinci grade de libertate. Controlul servomotoarelor este realizat prin semnale PWM generate de ESP32.

Componenta software este realizată folosind C++. Aplicația implementează un server web asincron care rulează direct pe ESP32, permițând o gestionare non-blocantă și rapidă a cererilor utilizatorului.

Interfața web reprezintă componenta de interacțiune cu utilizatorul și este dezvoltată utilizând tehnologii standard precum HTML, CSS și JavaScript. Aceasta este găzduită direct pe microcontroler și poate fi accesată prin intermediul unui browser web de pe orice dispozitiv conectat la aceeași rețea. Utilizatorul poate controla poziția degetelor folosind comenzi predefinite, ce sunt transmise în timp real către serverul web integrat. Comunicarea dintre frontend și backend este realizată cu ajutorul cererilor HTTP asincrone.

Arhitectura generală a acestui sistem pornește din browser, comunică cu serverul web ESP32, realizează controlul PWM și apoi mișcă servomotoarele. Această structură modulară separă componenta de interfață, cea de logică de control și cea de acționare hardware. În plus, datorită faptului că se folosește un server web, este eliminată necesitatea unei aplicații dedicate, oferind portabilitate și ușurință în utilizare.

Prin îmbinarea conceptelor din domeniul embedded, cel al comunicațiilor IoT și cel al programării web, această aplicație demonstrează modul în care un microcontroler cu resurse limitate poate susține un sistem interactiv în timp real. Proiectul evidențiază integritatea eficientă dintre hardware și software, oferind o soluție compactă, scalabilă și totuși ușor de utilizat pentru controlul unei mâini robotice.

Capitolul 1. Fundamente teoretice privind sistemele embedded și IoT

1.1 Arhitectura sistemelor embedded

Sistemele embedded reprezintă sisteme de calcul specializate, proiectate pentru a îndeplini funcții dedicate în cadrul unor aplicații. Spre deosebire de sistemele de calcul generale, acestea sunt optimizate pentru eficiență energetică, dimensiuni reduse, cost scăzut și execuție deterministă.

Această arhitectură constă în combinarea atât a componentelor hardware, cât și a componentelor software, create special pentru a îndeplini sarcini în mod eficient. Sistemele embedded au în principal două ramuri: hardware și software. Ramura hardware include componente precum microcontrolere, memorii, interfețe, senzori și surse de alimentare. Ramura software conține sistemul de operare, middleware și programe specifice aplicației.

În sistemele embedded, limbajul de programare poate fi ales în funcție de microcontroler și de tipul funcționalității pe care programatorul dorește să o implementeze. În cazul unei funcționalități relativ simple, fără restricții de memorie, se poate opta pentru programarea în Python, pe când în cazul în care memoria devine o problemă reală și este nevoie de acces direct la adrese și registre, se optează pentru folosirea limbajelor C sau C++.

În cadrul acestui proiect, sistemul embedded este construit în jurul unui microcontroler ESP32, produs de compania Espressif Systems, care integrează un procesor dual-core, memorie RAM și Flash, periferice digitale și analogice, conexiune Wi-Fi și Bluetooth, dar și suport pentru un sistem de operare în timp real (FreeRTOS).

Din punct de vedere al arhitecturii software, aplicația presupune organizarea codului în module funcționale distincte. Această structurare modulară crește lizibilitatea codului, facilitează mentenanța și permite extinderea ulterioară a sistemului.

1.2. Internet of Things (IoT)

Această categorie se referă la o rețea vastă de obiecte, pornind de la dispozitive de zi cu zi din casă până la dispozitive complexe utilizate în industrie, care sunt încorporate cu senzori, software și alte tehnologii. Aceste dispozitive se pot conecta la internet și pot comunica între ele, fapt ce le permite să colecteze și să împartă date fără intervenție umană.

Componentele cheie, fie ele hardware sau software, regăsite într-un dispozitiv IoT sunt în general senzori, conexiuni pentru comunicare, procesarea datelor și o interfață pentru utilizator. Aceste dispozitive IoT ajută la automatizarea task-urilor, luarea de decizii mai inteligente, optimizarea operațiunilor și îmbunătățirea experienței utilizatorilor.

Acest proiect poate fi considerat un dispozitiv IoT datorită integrării modulului Wi-Fi al microcontrolerului, caracteristică care permite controlul și monitorizarea sistemului de la distanță. Prin intermediul conexiunii, utilizatorul poate trimite comenzi către sistem, iar acesta răspunde prin trimiterea de date legate de starea servomotoarelor.

Din punct de vedere software, integrarea proiectului în domeniul IoT necesită implementarea unei arhitecturi de tip client-server. Microcontrolerul poate opera fie ca server web,

fie ca client. Datele colectate pot fi structurate în formate standardizate, precum JSON, făcând astfel posibilă integrarea ulterioară a acestora într-un sistem mai complex.

Deși, în forma curentă, proiectul nu automatizează anumite procese industriale, arhitectura permite totuși scalabilitate și dezvoltare ulterioară. Prin adăugarea unor funcționalități suplimentare și implementarea unor algoritmi mai avansați, sistemul poate evolua într-o platformă IoT mai complexă, aplicabilă în robotică educațională sau în dezvoltarea protezelor inteligente.

Capitolul 2. Prezentarea platformei ESP32

2.1. Arhitectura ESP32

În dezvoltarea sistemului de control pentru mâna robotică, a fost utilizată platforma ESP32, nu doar pentru proprietățile hardware, ci în special pentru flexibilitatea software. Această platformă este capabilă să gestioneze procese concurente, comunicație în rețea și control în timp real.

Microcontrolerul oferă suport pentru programare în C++ și rulează un sistem de operare în timp real (FreeRTOS), ceea ce permite dezvoltarea unei arhitecturi software modulare și scalabile. Astfel, în cadrul acestui proiect, accentul a fost pus pe organizarea logică a sistemului, separarea responsabilităților și optimizarea execuției programului. Din punct de vedere informatic, această platformă poate fi privită ca un sistem distribuit în miniatură, în care fiecare componentă software îndeplinește un rol bine definit.

Un element important al acestei platforme îl reprezintă sistemul cu două nuclee. Acestea permit execuția paralelă a mai multor sarcini, oferind posibilitatea implementării unui model concurent de procesare.

În cadrul sistemului dezvoltat, această caracteristică a microcontrolerului poate fi folosită pentru separarea funcționalităților în două categorii, câte una pentru fiecare core. Aceste categorii sunt: procesele de control în timp real, ce gestionează poziția degetelor, și procesele de comunicație și interfață, ce se ocupă de primirea comenzilor sau transmiterea stării sistemului.

Prin această separare a funcționalităților, se reduce riscul de blocare a sistemului și se menține un răspuns rapid. Această abordare respectă principiul separării responsabilităților (Separation of Concerns¹) și contribuie astfel la creșterea stabilității și clarității codului.

Figura atașată mai jos reprezintă diagrama de separare a funcționalităților, conform căreia fiecare modul al aplicației este responsabil de o anumită funcționalitate.

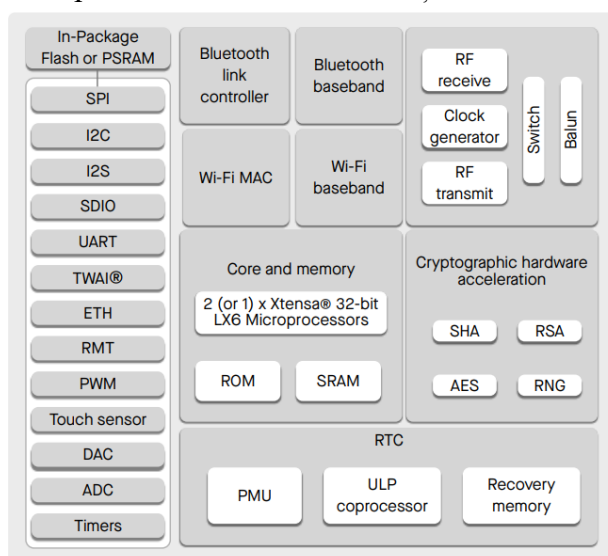


Figura 2.1. ESP32 Functional Block Diagram (Sursa bibliografică: [1])

¹ Principiu de proiectare care împarte programul în secțiuni distincte, astfel încât fiecare modul să gestioneze o singură funcție specifică fără a interfera cu celelalte.

Din analiza diagramei, se poate constata că sistemul este configurat pe mai multe niveluri distincte. În mijlocul figurii se poate observa unitatea de procesare, care conține unul sau două nuclee pe 32 de biți, responsabilă pentru executarea programului și a operațiilor. De asemenea, se pot observa și memoriile ROM și SRAM, utile pentru stocarea codului și a datelor necesare în timpul execuției.

Un alt aspect important al diagramei îl reprezintă modulele de comunicație wireless, Wi-Fi și Bluetooth. Acestea sunt alcătuite la rândul lor din mai multe subcomponente, dar rolul lor principal este de a permite comunicarea fără fir direct la nivel hardware.

Diagrama subliniază prezența numeroaselor interfețe periferice hardware, precum SPI, I2C, UART, PWM și ADC. Acestea permit microcontrolerului să se conecteze la diverse dispozitive externe și să controleze componentele hardware. În acest proiect, modulul PWM este esențial pentru controlul precis al servomotoarelor care mișcă degetele mâinii robotice. Astfel, microcontrolerul poate ajusta poziția fiecărui servomotor pe baza comenzilor primite de la utilizator.

Modulul Wi-Fi încorporat permite crearea unui server web direct pe microcontroler, facilitând comunicarea utilizator-sistem printr-un browser. Această funcționalitate face din ESP32 o platformă ideală pentru aplicații IoT, oferind control și monitorizare prin rețea.

2.2. Programarea în C++

Programarea microcontrolerului ESP32 poate fi realizată prin mai multe medii și framework-uri, cele mai frecvent utilizate fiind ESP-IDF și Arduino. În cadrul proiectului, a fost utilizat framework-ul Arduino, datorită ușurinței de utilizare și disponibilității unui număr mare de librării. Dezvoltarea aplicației s-a realizat în Visual Studio Code, folosind extensia PlatformIO, platformă care se ocupă cu gestionarea dependențelor, compilarea și încărcarea codului pe microcontroler. Limbajul de programare C++ a fost ales datorită suportului pentru programarea orientată pe obiect și eficienței ridicate în mediile embedded. Această abordare permite dezvoltatorului să se concentreze asupra logicii proiectului și asupra interacțiunii dintre componentele sistemului.

În comparație cu alte limbaje precum Python, C++ permite gestionarea memoriei și oferă performanțe superioare, proprietăți esențiale în proiectele embedded, unde limitările de memorie și viteza de execuție sunt critice. În același timp, programarea pe module și orientată pe obiect contribuie semnificativ la realizarea unei aplicații scalabile și ușor de întreținut.

Aplicațiile dezvoltate pentru ESP32 dispun de un număr mare de librării software, care oferă funcționalități predefinite pentru interacțiunea cu diferite componente hardware și pentru implementarea comunicației în rețea. Cu ajutorul acestor librării, dezvoltarea aplicației este semnificativ simplificată, deoarece dezvoltatorul poate folosi implementări optimizate pentru funcționalități complexe, precum comunicarea HTTP sau controlul semnalelor PWM.

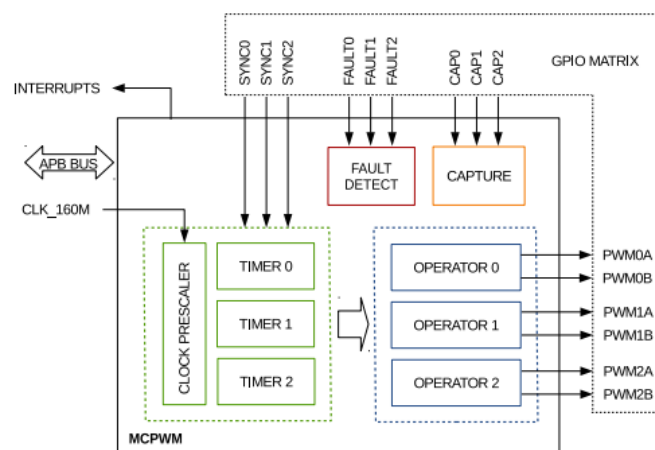


Figura 2.2. MCPWM Module Overview (Sursa bibliografică:[2])

Figura prezintă o vedere de ansamblu asupra modului MCPWM (Motor Control PWM²) integrat în ESP32, componentă esențială pentru generarea semnalelor PWM necesare controlului servomotoarelor. Acest modul include timere programabile (TIMER 0, TIMER 1, TIMER 2) sincronizate prin semnale SYNC, care stabilesc frecvența și perioada semnalelor PWM.

Controlul precis al servomotoarelor este realizat prin procesarea datelor temporale în unități dedicate (OPERATOR 0, 1 și 2), care generează semnalele PWM corespunzătoare. Suplimentar, integritatea sistemului este menținută prin modulele de detecție a erorilor (FAULT DETECT) și de captură a semnalelor (CAPTURE), asigurând monitorizarea activă și protecția în timp real a proceselor.

Prin utilizarea MCPWM, proiectul poate regla cu precizie unghiurile de rotație ale servomotoarelor, prin modificarea parametrilor semnalului PWM, cum ar fi duty cycle-ul, în timp real. Astfel, MCPWM asigură o interfață robustă și flexibilă între software și hardware, facilitând implementarea unui control performant și stabil al mâinii robotice. Formula de calcul a duty cycle-ului 0%-100% este:

$$Duty = \frac{(Period - A)}{Period} \quad (2.3)$$

Dacă A se potrivește cu valoarea timer-ului PWM și timer-ul PWM este în creștere, atunci ieșirea PWM este ridicată (pulled up). Dacă A se potrivește cu valoarea timer-ului PWM în timp ce timer-ul PWM este în scădere, atunci ieșirea PWM este coborâtă (pulled low).

Un rol important al proiectului implementat în C++ este realizarea interacțiunii dintre interfața software și componentele hardware. Programul primește de la utilizator, prin interfața web, valorile de poziție și le convertește în unghiuri de rotație.

Arhitectura modului MCPWM din cadrul microcontrolerului ESP32 se fundamentează pe trei timere hardware independente (TIMER0, 1 și 2), care constituie baza de timp pentru generarea semnalelor. Frecvența de operare este configurabilă prin divizarea semnalului de ceas

² pulse width modulator

al sistemului utilizând un prescaler³, permițând astfel adaptarea precisă a perioadei PWM la cerințele aplicației.

Funcționarea acestor timere se bazează pe unități de numărare (incrementare sau decrementare), ale căror valori sunt monitorizate constant de unitățile de comparare din cadrul operatorilor MCPWM. În timp ce timerele definesc perioada semnalului, operatorii gestionează factorul de umplere (duty cycle) prin raportarea contorului la praguri prestabilite.

Fiecare timer poate fi mapat către unul dintre cei trei operatori disponibili, care generează formele de undă pe ieșirile complementare (PWMxA și PWMxB), rutate ulterior către pinii fizici prin matricea GPIO.

Matematic, frecvența rezultată după prescaler este:

$$f_{timer} = \frac{f_{clock}}{prescaler} \quad (2.4)$$

La nivel software, comenzile recepționate via interfața web sunt procesate de un algoritm dezvoltat în C++, care actualizează parametrii de control în timp real. Acest flux asigură o latență minimă și o precizie ridicată datorită implementării serverului web asincron

³ circuit hardware care împarte frecvența semnalului de ceas al sistemului pentru a obține o frecvență mai mică

Capitolul 3. Servere web asincrone în sisteme embedded

3.1. Modelul sincron și asincron

În sistemele embedded, serverul web funcționează ca o interfață între utilizator și componentele hardware, folosind rețele de comunicație precum Wi-Fi. Performanța și capacitatea de extindere a acestui tip de sistem depind în mare măsură de modul în care sunt gestionate cererile venite de la utilizatori. În general, există două abordări principale, acestea fiind modelul sincron și modelul asincron.

În cadrul modelului sincron, cererile sunt procesate secvențial, una câte una. Fiecare operațiune trebuie finalizată complet înainte ca serverul să poată prelua o nouă comandă. Deși această metodă este simplă și ușor de implementat, ea poate deveni inefficientă în situațiile în care anumite cereri durează mai mult timp, cum ar fi citirea unor senzori complexi sau efectuarea unor calcule intensive. În astfel de cazuri, întregul sistem poate fi blocat temporar, ceea ce duce la întârzieri semnificative și afectează negativ experiența utilizatorului. Din acest motiv, modelul sincron nu este recomandat pentru aplicații care necesită reacții rapide sau gestionarea simultană a mai multor conexiuni.

Pe de altă parte, modelul asincron, cunoscut și ca model non-blocking, permite procesarea mai multor cereri simultan, fără a opri execuția principală a programului. Această arhitectură este deosebit de potrivită pentru microcontrolere precum ESP32, deoarece serverul poate răspunde imediat la cererile noi, în timp ce sarcinile care necesită mai mult timp se desfășoară în fundal. Astfel, procesele critice, cum este controlul precis al servomotoarelor care acționează degetele unei mâini robotice, nu sunt afectate de traficul de rețea sau de alte cereri concurente.

Implementarea unui server web asincron oferă avantaje importante în mediile embedded, unde resursele de memorie și puterea de procesare sunt limitate. Aceasta permite o scalabilitate ridicată, o utilizare mai eficientă a resurselor și o separare clară între logica de comunicație și controlul hardware. Rezultatul este un sistem stabil, modular și foarte receptiv, capabil să gestioneze multiple cereri fără a compromite performanța. Datorită acestor caracteristici, serverele web asincrone reprezintă soluția optimă pentru proiecte moderne de robotică și aplicații în timp real, unde viteza de răspuns și fiabilitatea sunt esențiale.

În cadrul proiectului dezvoltat, serverul web care controlează mâna robotică rulează pe ESP32 folosind un model asincron, ceea ce se aliniază perfect cu avantajele discutate anterior. Alegerea acestui model permite gestionarea simultană a mai multor cereri de la utilizator, fără a bloca funcțiile critice ale sistemului, cum ar fi controlul servomotoarelor ce mișcă degetele mâinii robotice.

Astfel, atunci când utilizatorul trimite comenzi prin interfața web, ESP32 poate procesa aceste cereri în timp real, în paralel cu monitorizarea și ajustarea poziției fiecărui servomotor. Această abordare non-blocking asigură că, indiferent de numărul de cereri primite sau de viteza de rețea, poziția degetelor este actualizată rapid și precis, fără întârzieri perceptibile.

Această structură transformă ESP32 într-un sistem distribuit de mici dimensiuni, unde fiecare segment software își îndeplinește funcția specifică fără a perturba activitatea celorlalte

module. Această abordare garantează stabilitatea și modularitatea întregului ansamblu, oferind un control exact asupra mecanismelor mâinii robotice și o interacțiune naturală cu utilizatorul, totul fără a depinde de aplicații externe sau de un consum excesiv de resurse.

Totodată, arhitectura asincronă asigură o bază solidă pentru dezvoltări ulterioare. Dacă pe parcurs apar cerințe noi, precum integrarea unor senzori adiționali sau a unor algoritmi mai sofisticăți, sistemul actual permite implementarea acestora fără a degrada viteza de reacție sau performanța generală a dispozitivului.

3.2. Evenimente și callback-uri

În programarea modernă, evenimentele și callback-urile sunt pilonii execuției asincrone, permițând sistemelor să reacționeze instantaneu la stimuli fără a întrerupe fluxul principal de lucru. Un eveniment funcționează ca un semnal digital, precum o comandă de la utilizator sau citirea unui senzor, care anunță că o anumită condiție a fost îndeplinită și necesită procesare.

Callback-urile reprezintă funcțiile de răspuns direct asociate acestor semnale. Ele se activează automat doar atunci când evenimentul are loc, eliminând necesitatea ca procesorul să verifice constant starea componentelor. Această abordare non-blocking este vitală pentru dispozitivele embedded cu resurse limitate, deoarece permite gestionarea eficientă a mai multor sarcini în paralel.

Astfel, utilizatorul primește un feedback instantaneu, deoarece sistemul nu mai „îngheață” în timp ce așteaptă finalizarea unei sarcini interne. Această fluiditate în execuție face ca interfața de control să pară naturală și receptivă, eliminând orice latență sesizabilă între comandă și mișcare.

Mai mult, acest model protejează integritatea sistemului în situații critice. Dacă un modul software întâmpină o întârziere, restul funcțiilor, cum ar fi oprirea de urgență sau monitorizarea senzorilor de siguranță, continuă să ruleze independent, garantând o operare sigură a brațului robotic în orice moment.

În cadrul proiectului, serverul de pe ESP32 aplică această logică pentru coordonarea mâinii robotice. Interacțiunea utilizatorului cu interfața web declanșează un eveniment, iar callback-ul corespunzător traduce imediat acea comandă în unghiuri de rotație și semnale PWM pentru servomotoare. În tot acest timp, microcontrolerul rămâne liber să monitorizeze alte funcții sau să preia noi cereri, asigurând o mișcare fluidă și precisă a degetelor robotice.

Această arhitectură garantează stabilitatea și reactivitatea sistemului, chiar și în condiții de utilizare intensă. Prin separarea clară a proceselor de comunicare, procesare și control hardware, se obține un ansamblu modular și scalabil. Rezultatul final este un sistem de robotică modern, capabil de performanță în timp real, unde toate componentele software și hardware colaborează armonios prin intermediul programării asincrone.

3.3. Gestionarea cererilor HTTP

În serverele web asincrone implementate pe microcontrolere precum ESP32, gestionarea cererilor HTTP reprezintă un element esențial pentru menținerea performanței și stabilității sistemului. Fiecare cerere HTTP primită de la client este tratată ca un eveniment independent, care declanșează un callback asociat, fără a bloca execuția buclei principale sau a altor procese critice.

Această abordare permite microcontrolerului să proceseze simultan mai multe cereri, menținând în paralel controlul precis al componentelor hardware, cum ar fi servomotoarele.

Procesul de gestionare a cererilor HTTP se fundamentează pe un model de procesare asincron, în care solicitarea clientului este recepționată și stocată într-un buffer dedicat, permițând eliberarea imediată a resurselor procesorului pentru sarcini prioritare de monitorizare și control hardware. Ulterior, fluxul de execuție activează funcțiile de tip callback corespunzătoare, care decodează pachetele de date și generează răspunsul adecvat, fie prin servirea paginilor web, fie prin actualizarea parametrilor senzorilor sau modificarea unghiulară a servomotoarelor.

Această arhitectură software oferă avantaje tehnice fundamentale pentru stabilitatea sistemului, începând cu operarea de tip non-blocking, care garantează că procesarea unei cereri individuale nu blochează execuția modulelor critice sau gestionarea altor solicitări concurente.

Eficiența sistemului este susținută de o scalabilitate ridicată, performanța rămânând constantă chiar și sub sarcini ridicate prin gestionarea optimă a firelor de execuție. Un aspect crucial al acestei abordări este decuplarea logicii de comunicație de bucla principală de control (main loop), fapt ce asigură un comportament determinist al sarcinilor hardware și elimină riscul de jitter⁴ în semnalele de comandă. În final, integritatea operațională este menținută prin protejarea funcțiilor vitale, precum monitorizarea senzorilor sau protocoalele de siguranță, împotriva latențelor induse de traficul de date, asigurând astfel o rulare neîntreruptă și sigură a întregului ansamblu robotic.

Timpul total perceput de utilizator de la apăsarea unui buton în interfața web până la inițierea mișcării poate fi descompus în trei componente independente, fiecare corespunzând unei etape distincte din lanțul de procesare:

$$T_{total} = T_{re\tau ea} + T_{callback} + T_{delay} \quad (3.1)$$

Astfel, arhitectura serverului web asincron permite ESP32 să răspundă în timp real la comenzi HTTP, să actualizeze poziția servomotoarelor și să gestioneze interacțiuni multiple simultan, fără a compromite stabilitatea sau performanța întregului sistem. În contextul controlului mâinii robotice, această strategie garantează o execuție precisă și rapidă, permițând utilizatorului o interfață web fluidă și reactivă.

⁴ diferența între momentul în care un eveniment ar fi trebuit să apară și momentul în care apare efectiv.

Capitolul 4. Proiectarea sistemului mâinii robotice

Proiectarea sistemului mâinii robotice reprezintă etapa crucială în care cerințele teoretice și funcționale identificate anterior se transformă în specificații concrete pentru implementare hardware și software. În această fază, se urmărește definirea clară a modului în care componentele sistemului interacționează, identificarea responsabilităților fiecărui modul și asigurarea unui flux de date eficient între interfața de control și mecanismul de acționare.

Obiectivul principal este crearea unui sistem modular, scalabil și fiabil, capabil să execute comenzi în timp real, să ofere feedback rapid utilizatorului și să mențină integritatea hardware-ului chiar și în situații critice. În continuare, capitolul se concentrează pe cerințele funcționale și nefuncționale, precum și pe arhitectura generală a sistemului, evidențiind fluxul de date și interacțiunea între browser, microcontrolerul ESP32 și servomotoarele care controlează degetele mâinii robotice.

4.1. Cerințe funcționale

Cerințele funcționale stabilesc parametrii operaționali ai sistemului, vizând controlul determinist și de înaltă precizie al structurii robotice. Sistemul este proiectat pentru a permite gestionarea independentă a fiecărui deget prin intermediul unor servomotoare dedicate, impunând o rezoluție unghiulară minimă de 1° pentru a facilita execuția unor mișcări fine. Performanța în timp real este esențială, arhitectura fiind optimizată pentru a minimiza latențele și a menține sincronizarea inter-digitală în timpul gesturilor complexe. Componenta de interfațare asigură o experiență intuitivă și responsivă, permițând monitorizarea și configurarea stării în timp real prin protocoale de comunicație de înaltă eficiență [3.3. Gestionarea cererilor HTTP]. Această abordare garantează un feedback instantaneu către utilizator, reflectând poziția curentă a fiecărui servomotor și validând vizual execuția comenzilor.

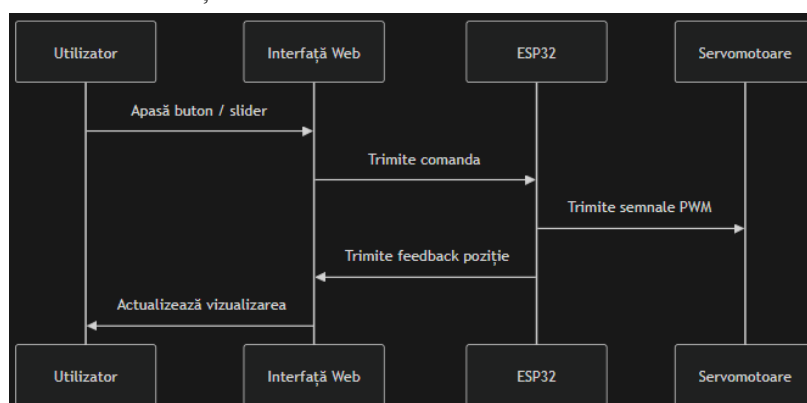


Figura 4.1. Diagrama de secvență

Când utilizatorul interacționează cu interfața web, prin apăsarea unui buton sau ajustarea unui slider pentru un deget, comanda este captată și este trimisă către ESP32.

Unitatea centrală de procesare recepționează și decodează pachetele de date, calculând parametrii semnalelor PWM necesari acționării fiecărui deget al structurii robotice. Aceste semnale sunt direcționate către pinii de ieșire configurați, declanșând translația unghiulară a servomotoarelor conform setărilor recepționate. Grație arhitecturii de procesare paralele, ESP32

gestionează simultan sarcini secundare, precum monitorizarea senzorilor de siguranță și diagnoza sistemului, menținând un flux de execuție determinist și eliminând blocajele software în timpul mișcărilor complexe.

Finalizarea secvenței de mișcare este urmată de transmiterea unui flux de feedback către interfața web, asigurând sincronizarea datelor între starea fizică a dispozitivului și reprezentarea sa virtuală. Această buclă de control bidirecțională permite actualizarea instantanee a interfeței utilizatorului, oferind o monitorizare precisă a poziției fiecărui segment digital. Implementarea acestui mecanism garantează nu doar o acționare simultană și fluidă a degetelor, ci și o experiență de utilizare sigură, bazată pe un răspuns rapid al sistemului la comenzile operatorului.

Din perspectivă computațională, sistemul suportă execuția simultană a comenzilor pentru întregul ansamblu de servomotoare, evitând blocarea microcontrolerului [3.2. Evenimente și callback-uri].

Arhitectura este concepută pentru a fi modulară și scalabilă, facilitând integrarea ulterioară de noi senzori sau unități de execuție fără restructurarea nucleului software. Schimbul de date între interfața web și ESP32, asigură o interoperabilitate ridicată și stabilitate în extinderea funcționalităților sistemului fără a compromite resursele de calcul disponibile.

4.2. Cerințe nefuncționale

Pe lângă cerințele funcționale care definesc comportamentul operațional al sistemului, proiectarea unei aplicații mecatronice implică și stabilirea unor cerințe nefuncționale. Acestea descriu caracteristicile de performanță, fiabilitate și robustețe ale sistemului, influențând direct experiența utilizatorului și stabilitatea funcționării pe termen lung.

În cazul sistemelor robotice controlate prin rețea, două dintre cele mai importante aspecte nefuncționale sunt timpul de răspuns și stabilitatea comunicației dintre interfața de control și unitatea de procesare.

4.2.1. Timpul de răspuns

Timpul de răspuns reprezintă intervalul dintre momentul în care utilizatorul transmite o comandă și momentul în care efectul acesteia devine vizibil la nivelul mecanismului acționat. În aplicațiile robotice interactive, acest parametru trebuie menținut la valori reduse pentru a asigura o experiență de control intuitivă și predictibilă.

Un timp de răspuns ridicat poate genera întârzieri perceptibile între comandă și execuția mișcării, ceea ce poate conduce la dificultăți în manipularea precisă a sistemului robotic. Din acest motiv, arhitectura software și infrastructura de comunicație trebuie optimizate astfel încât procesarea comenzilor să fie realizată într-un mod eficient, fără blocaje sau întârzieri semnificative.

Pentru a putea verifica în mod riguros îndeplinirea acestei cerințe în cadrul testelor de performanță, timpul de răspuns al sistemului se formalizează ca suma componentelor sale măsurabile:

$$T_{raspuns} = T_{wireless} + T_{procesare} + T_{PWM} \quad (4.2)$$

4.2.2. Stabilitatea comunicației

În arhitectura sistemelor încorporate interconectate, stabilitatea fluxului de date constituie un pilon fundamental al procesului de proiectare, influențând direct predictibilitatea comportamentului robotic. Interacțiunea dintre interfața web și segmentul hardware necesită un grad ridicat de fiabilitate pentru a garanta integritatea comenzilor transmise, în special în prezența latențelor variabile sau a fluctuațiilor specifice mediului de transmisie wireless.

Orice întrerupere în recepția pachetelor de date poate duce la pierderea instrucțiunilor sau la execuția acestora cu decalaje temporale critice, compromițând sincronizarea cinematică a mecanismului.

Pentru a răspunde acestor cerințe nefuncționale, proiectul exploatează subsistemul de comunicație integrat al microcontrolerului **ESP32**, stabilind o conexiune în cadrul unei rețele locale (**LAN**). Această abordare minimizează rutele de transmisie și optimizează timpul de răspuns, asigurând un transfer de date cvasi-instantaneu între punctul de control și unitatea de execuție.

Complementar, arhitectura software implementează un model de procesare non-blocantă, care izolează sarcinile de comunicație de funcțiile hardware critice. Astfel, gestionarea stivelor de rețea nu interferează cu generarea semnalelor PWM, garantând menținerea preciziei poziționale și a fluidității mișcărilor, indiferent de intensitatea traficului de date de intrare.

Pentru a menține stabilitatea sistemului în timpul funcționării, mecanismele de control software sunt organizate într-un mod care permite gestionarea simultană a mai multor sarcini. Această abordare asigură continuitatea proceselor critice și previne blocarea microcontrolerului în cazul unor solicitări intense sau al unor variații temporare ale traficului de rețea. Prin urmare, sistemul poate răspunde rapid comenzilor utilizatorului, menținând în același timp o funcționare stabilă și predictibilă a mecanismului robotic.

Respectarea riguroasă a acestor criterii nefuncționale garantează o interacțiune deterministă și fluidă între operator și ansamblul robotic, eliminând incertitudinile operaționale cauzate de latențele de rețea. Această stabilitate a fluxului de date și a timpului de răspuns constituie fundamentul necesar pentru implementarea structurii sistemice complexe, a cărei arhitectură generală și mod de interconectare a modulelor componente sunt detaliate în subcapitolul următor.

4.3. Arhitectura generală a sistemului

Arhitectura unui sistem robotic reprezintă structura conceptuală care definește modul în care componentele hardware și software sunt organizate și interconectate pentru a realiza funcționalitatea dorită. În sistemele moderne de control distribuit, arhitectura este concepută astfel încât să asigure separarea clară a responsabilităților între diferitele module ale sistemului, facilitând dezvoltarea, întreținerea și extinderea ulterioară a acestuia.

Sistemele robotice moderne, guvernate prin interfețe digitale, adoptă o structură organizațională stratificată, menită să separe nivelurile de abstractizare ale controlului. În acest model ierarhic, nivelul superior (aplicația) este reprezentat de interfața utilizatorului, având rolul critic de achiziție a comenzilor și de monitorizare a stării sistemului.

Nivelul intermediar, sau unitatea de procesare logică, acționează ca un traducător între intenția utilizatorului și execuția fizică, transformând instrucțiunile de nivel înalt în semnale de control deterministe. În final, nivelul inferior cuprinde subsistemul de execuție, unde actuatoarele și componentele cinematice convertesc energia electrică în lucru mecanic.

În cadrul proiectului de față, arhitectura este segmentată în trei piloni funcționali: interfața web de operare, unitatea de control bazată pe microcontrolerul ESP32 și mecanismul de acționare al structurii antropomorfe.

Interfața web constituie punctul de convergență între operator și sistem, oferind un mediu grafic intuitiv pentru configurarea vectorilor de poziție ai fiecărui deget. Prin intermediul protocolului de comunicație, aceste setări sunt încapsulate în cereri de rețea și transmise către unitatea centrală.

Microcontrolerul ESP32 exercită funcția de hub decizional, fiind responsabil pentru interceptarea pachetelor de date, decodificarea parametrilor unghiulari și generarea secvențelor de comandă adecvate. Prin intermediul interfeței de comunicație I²C, ESP32 transmite comenzile calculate către modulul driver PCA9685, un controler PWM dedicat cu 16 canale independente. Acesta preia responsabilitatea generării semnalelor PWM cu factorul de umplere (Duty Cycle) corespunzător fiecărui actuator, eliminând astfel încărcarea perifericelor interne ale microcontrolerului și oferind o scalabilitate superioară în controlul servourilor..

La baza acestei ierarhii se regăsește subsistemul mecanic, definit prin structura articulată a degetelor și rețeaua de transmisie prin tendoane artificiale. Servomotoarele convertesc impulsurile electrice în mișcare rotațională, forță care este ulterior tradusă în tracțiune liniară prin intermediul firelor de înaltă rezistență. Această tensiune mecanică determină flexia segmentelor digitale, replicând cinematica mâinii umane și facilitând manipularea obiectelor prin mișcări de prehensiune coordonate.

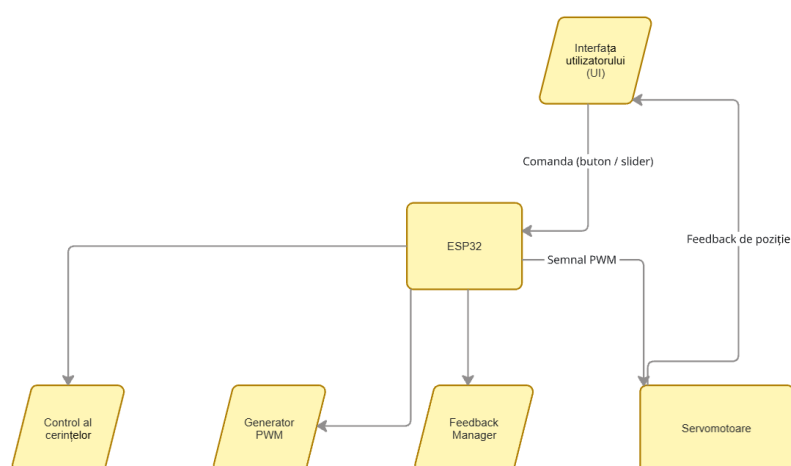


Figura 4.3. Diagrama fluxului proiectului

Diagrama ilustrează structura generală a sistemului, evidențiind principalele componente hardware și software și modul în care acestea comunică pentru a coordona funcționarea mâinii robotice corespunzător.

La baza interacțiunii se află Interfața utilizatorului (UI), care permite operatorului să transmită comenzi prin intermediul unor butoane sau slider-e. Aceste comenzi sunt trimise către microcontrolerul ESP32, care reprezintă unitatea centrală de procesare și control a sistemului.

În cadrul ESP32, comenzile primite sunt gestionate de modulul de Control al cerințelor, care interpretează intențiile utilizatorului și determină acțiunile necesare pentru fiecare deget al mâinii robotice. Pe baza acestor date, Generatorul PWM produce semnale modulare în lățime de impuls (PWM), care sunt transmise către servomotoare pentru a regla poziția unghiulară a fiecărui deget.

În paralel, modulul Feedback Manager preia date de poziție de la servomotoare și le transmite înapoi către interfața utilizatorului, închizând astfel bucla de control. Acest feedback permite monitorizarea în timp real a stării sistemului, oferind utilizatorului o reprezentare vizuală actualizată a poziției degetelor robotice.

Astfel, această arhitectură modulară și bidirecțională asigură o interacțiune fluentă și precisă între utilizator și sistem, garantând răspunsuri rapide la comenzi și menținerea sincronizării între starea fizică a mâinii robotice și reprezentarea sa virtuală.

Fluxul de date al sistemului urmează astfel un traseu bine definit: comenzile utilizatorului sunt generate în interfața web, transmise prin intermediul browserului către microcontrolerul ESP32 și convertite în semnale electrice de control pentru servomotoare. Aceste semnale produc mișcarea mecanică a degetelor, completând lanțul de interacțiune dintre utilizator și sistemul robotic.

Această arhitectură asigură o separare clară între nivelul de control software și nivelul mecanic de execuție, facilitând dezvoltarea și extinderea ulterioară a sistemului.

Capitolul 5. Implementare hardware

Implementarea hardware reprezintă etapa critică în care modelul teoretic al sistemului robotic este transpus într-o configurație fizică funcțională capabilă să execute operații complexe. În cadrul acestui proiect a fost realizată o structură antropomorfă proiectată pentru a replica funcțiile de prehensiune și manipulare specifice mâinii umane.

5.1. Generalizare și aplicabilitatea industrială

Servomotoarele reprezintă dispozitive electromecanice de precizie care integrează un motor electric, un sistem de transmisie prin angrenaje reductoare și un mecanism de retroacțiune pozițională într-o unitate compactă.

Spre deosebire de motoarele electrice convenționale, servomotoarele permit un control riguros asupra poziției unghiulare sau liniare, vitezei și accelerației axului de ieșire prin utilizarea unei bucle de control închise. Această capacitate de a menține o poziție specifică sub sarcină și de a răspunde rapid la variațiile semnalului de comandă le face indispensabile în automatizările moderne.

În sectorul industrial, servomotoarele sunt utilizate extensiv în sistemele de fabricație asistate de calculator, în special în brațele robotice articulate unde precizia și repetabilitatea sunt critice pentru procesele de asamblare. De asemenea, acestea joacă un rol fundamental în mașinile cu comandă numerică pentru poziționarea capetelor de tăiere și în liniile de ambalare automată unde viteza de reacție determină direct randamentul producției.

Alte aplicații notabile includ sistemele de orientare a panourilor fotovoltaice, controlul suprafețelor de zbor în industria aero-spațială și în vehiculele autonome unde sunt necesare corecții direcționale precise și sigure.

Servomotorul MG996R este un actuator de tip high-torque⁵ utilizat pe scară largă în prototiparea sistemelor robotice datorită raportului optim între performanța mecanică și dimensiunile constructive. Acesta reprezintă o evoluție a modelului precedent MG995, oferind o stabilitate îmbunătățită și un sistem de angrenaje metalice care îi conferă o rezistență sporită la uzură și la sollicitări mecanice intense.

În proiectele de anvergură redusă sau în prototiparea rapidă, servomotoarele sunt utilizate predominant ca unități de acționare fundamentale pentru structuri diverse, variind de la roboți bipezi și hexapozi până la brațe robotice articulate. Versatilitatea lor este demonstrată și în industria jucăriilor cu comandă radio (RC), unde constituie componenta principală a sistemelor de direcție datorită preciziei și fiabilității mecanice.

Un aspect esențial al acestor actuatoare este capacitatea de a asigura un control riguros al poziției fără a necesita sisteme externe de feedback complexe, facilitând astfel implementarea în mecanisme unde spațiul și resursele de calcul sunt limitate. Totodată, masa redusă a acestor dispozitive le face ideale pentru integrarea în roboți cu numeroase grade de libertate (DOF),

⁵ arhitectura sa internă este optimizată pentru a genera o forță de tracțiune sau apăsare considerabilă, sacrificând uneori viteza maximă de rotație în favoarea puterii brute.

precum structurile humanoide, unde minimizarea inerției segmentelor mobile este critică pentru menținerea echilibrului și a fluidității mișcărilor.

Din punct de vedere electric, dispozitivul operează într-un domeniu de tensiune cuprins între 4.8V și 7.2V, parametru ce influențează direct cuplul de blocare care poate atinge valori de până la 11 kg/cm la tensiunea maximă de alimentare.

Viteza de operare a acestui model este de aproximativ 0.17 secunde pentru o rotație de 60 de grade, asigurând o dinamică adecvată pentru mișcările de prehensiune. Controlul pozițional este realizat prin modulația în lățime a impulsurilor (PWM), unde durata unui impuls de 1.5 milisecunde corespunde de obicei poziției centrale de 90 de grade [11].

Construcția internă include un rulment dublu pentru axul principal, fapt ce minimizează frecarea și jocul mecanic, asigurând o durată de viață prelungită în regim de funcționare continuă.

Această combinație de cuplu ridicat și precizie digitală face din MG996R o componentă ideală pentru acționarea degetelor unei mâini robotice, unde forța de tracțiune necesară tensionării cablurilor este considerabilă.

Nucleul tehnologic al acestui proiect este reprezentat de platforma ESP32, un sistem pe un singur cip (SoC) de înaltă performanță care integrează conectivitate wireless și o putere de calcul dual-core superioară microcontrolerelor convenționale. Capacitatea sa de a gestiona simultan fluxuri de date complexe și periferice hardware dedicate pentru controlul mișcării îl definește ca o soluție optimă pentru coordonarea în timp real a întregului ansamblu robotic.

O analiză aprofundată a specificațiilor tehnice, a modului de gestionare a memoriei și a capacităților periferice ale acestui dispozitiv este prezentată în secțiunea 2.1. Arhitectura ESP32.

5.2. Integrarea componentelor și construcția prototipului

Proiectul prezent are ca scop dezvoltarea unui sistem robotic, conceput pentru a reproduce mișcările biomecanice ale mâinii umane prin utilizarea tehnologiilor moderne de fabricație aditivă și a unor metode eficiente de control electronic. Obiectivul principal al lucrării îl reprezintă realizarea unei structuri mecanice capabile să execute mișcări coordonate de flexie și extensie ale degetelor, controlate prin intermediul unei interfețe web intuitive care comunică direct cu microcontrolerul ESP32.

Construcția mecanică a sistemului se bazează pe un mecanism de tip tendon artificial, realizat din fire rezistente la tracțiune, care sunt tensionate de un ansamblu format din cinci servomotoare MG996R. Aceste servomotoare sunt amplasate astfel încât fiecare dintre ele să fie responsabil pentru acționarea unui deget, permițând astfel controlul individual al mișcărilor.

Funcționarea sistemului se bazează pe un model de procesare asincronă a comenzilor, ceea ce permite modificarea în timp real a unghiurilor de poziționare ale degetelor fără a întrerupe celelalte procese care rulează pe microcontroler. Această abordare contribuie la obținerea unei mișcări fluide și la menținerea unei reacții rapide a sistemului la comenzile utilizatorului.

Integrarea tuturor componentelor într-o arhitectură unitară conduce la realizarea unei platforme versatile, potrivite pentru studii și aplicații în domeniul protezelor robotice sau al manipulării obiectelor la distanță.

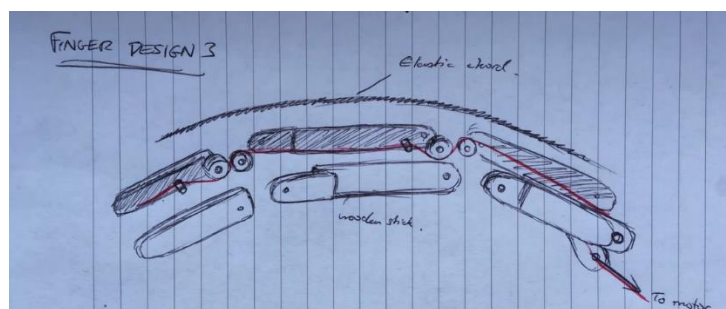


Figura 5.1. Robotic Hand finger design

Figura prezintă schema conceptuală a mecanismului unui deget al mâinii robotice, evidențiind modul în care mișcarea de flexie este realizată prin intermediul unui sistem de tendon artificial. Structura degetului este alcătuită din mai multe segmente articulate care imită falangele unui deget uman, fiecare segment fiind conectat prin articulații mecanice ce permit rotirea relativă între componente.

De-a lungul părții superioare a structurii este amplasat un element elastic, care are rolul de a asigura revenirea degetului în poziția inițială atunci când tensiunea exercitată de mecanismul de acționare este eliberată.

Mișcarea de flexie este realizată prin intermediul unui fir de acționare conectat la un servomotor, indicat în schemă prin direcția de tragere. În momentul în care servomotorul aplică forță asupra firului, acesta transmite tensiunea de-a lungul articulațiilor, determinând rotirea segmentelor și curbare progresivă a degetului.

În absența acestei forțe, elasticul întins generează o forță de revenire care readuce segmentele în poziția extinsă. Acest mecanism simplificat permite obținerea unei mișcări apropiate de cea naturală a degetelor umane, utilizând un număr redus de componente mecanice și un singur actuator pentru fiecare deget.

Configurația detaliată a canalelor și corespondența acestora cu elementele efectoare sunt sintetizate în tabelul de mai jos.

Componenta Efectoare	Canal PCA9685
Deget mic	Canal 4
Deget inelar	Canal 3
Deget mijlociu	Canal 2
Deget arătător	Canal 1
Deget mare	Canal 0 & Canal 5

Tabelul 5.2. Alocarea canalelor PCA9685 și codificarea cromatică a semnalelor de control

Sistematizarea conexiunilor hardware reprezintă o etapă esențială în asigurarea mentenanței și a fiabilității sistemului, facilitând totodată procesele de dezvoltare și depanare (debugging). Comunicația dintre ESP32 și modulul driver PCA9685 se realizează prin protocolul

I²C, utilizând pinii GPIO 21 (SDA) și GPIO 22 (SCL) ai microcontrolerului. Această abordare reduce semnificativ numărul de conexiuni fizice față de o soluție bazată pe pini GPIO dedicați, întregul flux de control fiind transmis printr-o singură magistrală serială.

Un element distinctiv al acestei configurații este utilizarea unui cod al culorilor pentru conductorii de semnal, metodă care reduce riscul erorilor de asamblare și optimizează intervențiile ulterioare asupra infrastructurii hardware.

Această structură de organizare nu doar că simplifică documentarea tehnică, dar asigură și o trasabilitate clară a fluxurilor de date de la unitatea de procesare către unitățile de execuție.

Prin alocarea unică a fiecărui deget unui canal dedicat al PCA9685, se obține un control granular și independent asupra mișcărilor, eliminând interferențele logice în procesul de generare a semnalelor PWM. Modulul PCA9685 preia generarea hardware a semnalelor PWM pentru toate cele 6 servouri simultan, descărcând astfel resursele interne ale ESP32.

Prin combinarea unei structuri hardware robuste cu o interfață de control accesibilă prin intermediul rețelei, sistemul demonstrează o metodă eficientă de implementare a controlului mecatronic pentru un mecanism robotic inspirat din anatomia mâinii umane.

Performanța funcțională a prototipului nu depinde exclusiv de calitatea componentelor mecanice utilizate sau de acuratețea procesului de asamblare, ci mai ales de interacțiunea strânsă dintre structura hardware și arhitectura logică a sistemului. Ansamblul mecatronic realizat constituie infrastructura fizică a sistemului, însă comportamentul dinamic și funcționalitatea acestuia sunt determinate de algoritmi de control implementați la nivelul firmware-ului.

În acest context, complexitatea mișcărilor și gradul de precizie al reacțiilor sistemului sunt rezultatul unei logici de programare bine definite, capabile să transforme comenzile utilizatorului în semnale electrice sincronizate, transmise către actuatoare. Această relație de interdependență dintre hardware și software permite coordonarea eficientă a tuturor componentelor și asigură executarea mișcărilor într-un mod stabil și predictibil.

Prin urmare, funcționarea optimă a sistemului se bazează pe această integrare dintre componenta fizică și cea logică, în care hardware-ul răspunde prompt comenzilor generate de software. Mecanismele de procesare, metodele de control și protocoalele de comunicare care permit realizarea acestei interacțiuni vor fi prezentate în detaliu în capitolul următor, dedicat implementării software a sistemului.

Capitolul 6. Implementare software

În paradigma ecosistemelor IoT (Internet of Things), arhitectura software constituie elementul de coeziune care transformă ansamblurile de senzori și actuatori în sisteme cibernetice inteligente, capabile de procesare și comunicare bi-direcțională în timp real. Conform principiilor fundamentate în secțiunea 1.2. Internet of Things (IoT), aceste sisteme facilitează convergența dintre mediul fizic și cel digital, oferind mecanisme avansate pentru monitorizarea și controlul automatizat al proceselor complexe.

O aplicație de tip embedded, optimizată pentru mediul IoT, este structurată ierarhic pe niveluri funcționale distincte: un nivel hardware destinat interfațării directe și execuției, un nivel de comunicație responsabil pentru managementul fluxurilor de date între nodurile rețelei și un nivel logic de control care guvernează comportamentul sistemului și interacțiunea cu utilizatorul.

Această stratificare modulară nu doar că optimizează procesele de dezvoltare și mentenanță, dar asigură și o scalabilitate ridicată, permițând integrarea de noi funcționalități fără a altera stabilitatea nucleului operațional.

În cadrul prezentului proiect, software-ul dezvoltat pentru controlul mâinii robotice adoptă această filosofie modulară, delimitând riguros algoritmi de pilotare a servomotoarelor, serviciile de web-hosting⁶ și logica de procesare asincronă.

Această demarcare clară a responsabilităților software garantează o lizibilitate sporită a codului sursă și o robustețe necesară pentru operarea într-un mediu de rețea dinamic, asigurând un răspuns determinist al sistemului la comenzile operatorului.

6.1. Structura generală a aplicației

Proiectarea aplicației software pentru controlul mâinii robotice a fost realizată prin separarea logică a responsabilităților între interfața de utilizator și motorul de procesare central, asigurând astfel o interacțiune coerentă între mediul digital și cel fizic. Segmentul de frontend este construit pe baza tehnologiilor web fundamentale, utilizând HTML pentru definirea structurii documentului, CSS pentru aplicarea regulilor de stilizare vizuală și JavaScript pentru implementarea comportamentului interactiv în timp real.

Această abordare permite operatorului să acceseze o consolă de control intuitivă, capabilă să genereze vectori de mișcare pentru fiecare segment digital al mâinii, să monitorizeze în timp real starea actuatorilor și să ajusteze parametrii de configurare fără a interacționa direct cu codul sursă.

Deși prezentul capitol oferă o privire de ansamblu asupra acestei componente, o analiză mai amplă a elementelor de design și a logicii de scripting din spatele interfeței grafice este detaliată în secțiunea 6.5. Structura HTML, stilizare CSS și funcționalitate JavaScript a lucrării.

În contrapondere cu interfața grafică, componenta de backend reprezintă nucleul computațional al sistemului, fiind dezvoltată în limbajul C++ și organizată după principii stricte de modularitate.

⁶ procesul de stocare și servire a fișierelor care compun interfața de utilizator

Unitatea principală de decizie mecatronică este localizată în modulul ‘hand_gestures.cpp’, unde sunt centralizate toate funcțiile matematice și logice necesare pentru interpretarea comenzilor primite și transpunerea acestora în mișcări precise ale servomotoarelor, gestionând în mod unitar bibliotecile de gesturi și limitele de cursă ale fiecărui deget.

Coordonarea fluxului principal de execuție este realizată în cadrul fișierului ‘main.cpp’, care implementează structura standard de operare a microcontrolerului prin intermediul funcțiilor de inițializare și al buclei principale de rulare.

În spiritul unei bune practici de programare, aceste funcții sunt menținute la un nivel de complexitate minim, rolul lor fiind limitat la delegarea sarcinilor către modulele specializate și supravegherea stării generale a sistemului. Toți parametrii critici de funcționare, inclusiv alocarea pinilor de intrare-ieșire pentru servomotoare, constantele de timp și setările de rețea, sunt centralizați în fișierul de configurare ‘main_cfg.hpp’, oferind un punct unic de control pentru adaptarea rapidă a hardware-ului fără modificarea logicii de control.

Integrarea dintre mediul web și hardware-ul propriu-zis este asigurată de modulul ‘web_server_connect.cpp’, care implementează arhitectura unui server web asincron capabil să gestioneze simultan cererile HTTP și sincronizarea bidirecțională a datelor.

Acest modul acționează ca un pod de comunicație, asigurând că instrucțiunile transmise prin intermediul browserului ajung la unitatea de control într-un format prelucrabil, menținând în același timp o latență minimă.

Această strategie de dezvoltare modulară transformă backend-ul într-o platformă robustă și scalabilă, unde separarea clară între configurațiile hardware, serverul de comunicație și logica gestuală simplifică semnificativ procesele de testare și mentenanță, oferind totodată flexibilitatea necesară pentru viitoare extinderi funcționale ale brațului robotic.

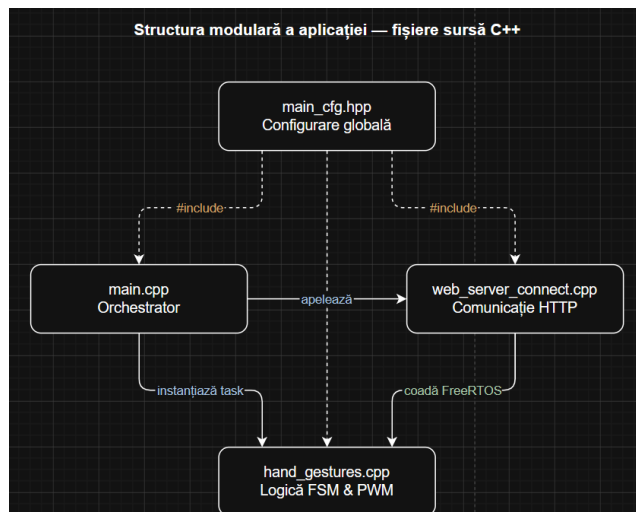


Figura 6.1. Structura modulară a aplicației

Diagrama sintetizează arhitectura modulară a backend-ului, ilustrând dependențele de compilare și fluxurile de apel dintre cele patru fișiere sursă ale proiectului. Fișierul `main_cfg.hpp` ocupă poziția centrală de configurare, fiind inclus de toate celelalte module și constituind singurul punct în care modificările de hardware, realocarea canalelor PCA9685 sau ajustarea constantelor de timp, se propagă automat în întreaga aplicație fără a atinge logica de control. `main.cpp`

îndeplinește exclusiv rolul de orchestrator: apelează `connect_to_server()` și `setup_routes()` din modulul de comunicație, instanțiază `servo_task` pe Core 1 prin `xTaskCreatePinnedToCore()` și lasă funcția `loop()` goală în mod deliberat, menținând punctul de intrare la complexitate minimă.

Modulul `web_server_connect.cpp` acționează ca pod între mediul web și hardware, interceptând cererile HTTP din browser și plasând comenzile în cozile FreeRTOS, de unde `hand_gestures.cpp` le preia și le transpune în semnale MCPWM către cele cinci servomotoare MG996R prin intermediul mașinii de stări finite descrise în subcapitolul următor.

6.2. Implementarea serverului web asincron

Serverul web constituie pilonul central de comunicație între operator și ansamblul robotic, reprezentând mediul prin care instrucțiunile digitale sunt translatate în acțiuni mecanice de precizie. În cadrul acestui proiect, funcționalitatea este asigurată de modulul ‘`web_server_connect.cpp`’, care integrează un server web asincron rulat direct pe microcontrolerul ESP32.

Această alegere tehnologică se bazează pe avantajele descrise în secțiunea 3.1. Modelul sincron și asincron, unde a fost evidențiat faptul că un model asincron permite sistemului să rămână receptiv la cererile de rețea fără a suspenda procesele de execuție din bucla principală.

Caracteristica definitorie a acestei implementări este capacitatea de operare non-blocantă, un aspect fundamental în ingineria sistemelor de tip Internet of Things. Spre deosebire de serverele web tradiționale, care procesează cererile în mod secvențial și pot suspenda execuția programului principal, modelul asincron permite gestionarea simultană a mai multor conexiuni fără a întrerupe procesele critice de generare a semnalelor PWM pentru servomotoare.

Din punct de vedere al implementării, acest comportament este realizat prin instanțierea unui obiect `AsyncWebServer server(80)`, care preia sarcina gestionării protocolului HTTP pe portul standard. Pentru a minimiza latența în mediul wireless, funcția `connect_to_server()` utilizează instrucțiunea `WiFi.setSleep(false)`, dezactivând modul de economisire a energiei al modemului radio, asigurând astfel un timp de răspuns optimizat pentru aplicații de control în timp real.

Separarea hardware a sarcinilor pe microcontrolerul ESP32 este realizată prin distribuirea controlată a proceselor între cele două nuclee disponibile. Primul nucleu, Core 0, preia în totalitate managementul comunicației wireless, fiind responsabil de stiva WiFi și de serverul web asincron; această configurare permite interceptarea și tratarea solicitărilor HTTP în fundal, eliminând interferențele cu operarea perifericelor. În paralel, Core 1 este rezervat exclusiv pentru rularea `servo_task`, o sarcină instanțiată prin `xTaskCreatePinnedToCore()` cu prioritatea 2. Acest task execută neîntrerupt funcția `update_gesture()` și generează semnalele PWM necesare. Excluderea altor procese native Espressif de pe Core 1 garantează că această buclă de control dispune de resurse dedicate complete la fiecare ciclu de 8 ms, fiind protejată integral de riscul preempțiunii cauzate de traficul de rețea.

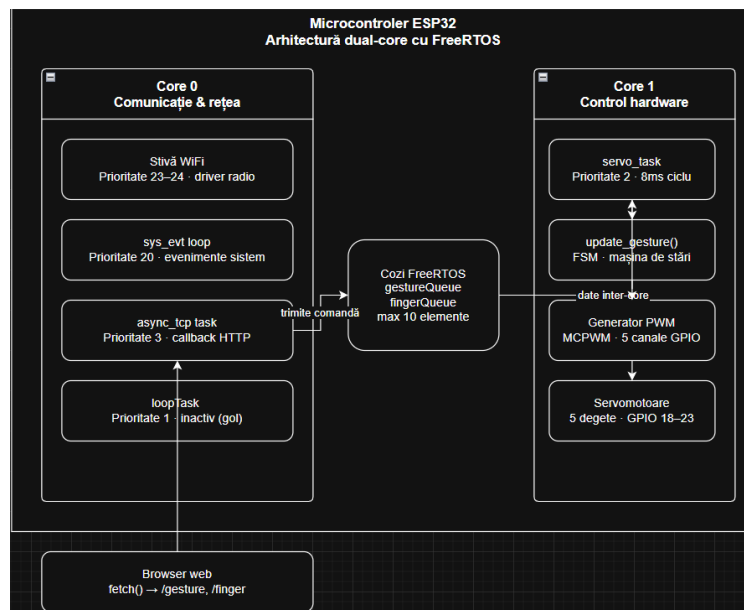


Figura 6.2. Arhitectura dual-core al ESP32 cu FreeRTOS

Diagrama ilustrează modul în care sarcinile software sunt distribuite între cele două nuclee ale microcontrolerului ESP32 în vederea garantării unui comportament determinist al sistemului.

Gestiunea execuției concurente în FreeRTOS se bazează pe o ierarhie strictă de priorități alocate fiecărui proces în parte. Astfel, pe Core 0 sunt concentrate sarcinile critice de rețea: driverul WiFi și timerul intern ocupă cel mai înalt nivel de prioritate din sistem (23-24), urmate de bucla de evenimente `sys_evt` la prioritatea 20 și de task-ul `async_tcp` la prioritatea 3, care se ocupă de callback-urile serverului asincron. Tot pe Core 0 se află și `loopTask` din framework-ul Arduino, configurat cu prioritatea 1, dar care rămâne inactiv și nu consumă resurse CPU deoarece este lăsat gol.

În contrast, Core 1 găzduiește exclusiv componenta hardware prin `servo_task`, setat la prioritatea 2. Această valoare este mai mare decât cea a task-ului `Idle`, dar se situează intenționat sub nivelul proceselor de sistem de pe celălalt nucleu. O asemenea arhitectură asigură că logica de control nu blochează stiva de rețea, permițând scheduler-ului să ofere controlul deplin către `servo_task` imediat ce intervalul de 8 ms din `vTaskDelay()` s-a scurs, fără riscul de a fi întrerupt de alte activități de pe același nucleu.

Logica de funcționare a serverului este structurată pentru a gestiona eficient diverse tipuri de interacțiuni protocolare prin definirea unor rute specifice în funcția `setup_routes()`. Un aspect esențial al implementării este utilizarea sistemului de fișiere LittleFS pentru a servi resursele statice ale interfeței (HTML, CSS, JavaScript), eliminând necesitatea stocării acestora sub formă de șiruri de caractere în memoria RAM.

Comunicarea activă se realizează prin ruta `/gesture`, unde serverul interceptează cererile de tip `HTTP_GET`. Atunci când interfața transmite un parametru, cum ar fi numele unui gest specific, serverul execută un callback care identifică instrucțiunea și o plasează în coada FreeRTOS, de unde `servo_task` o preia și inițiază secvența de mișcare corespunzătoare.

Această diviziune a sarcinilor permite menținerea unui timp de răspuns minim, eliminând latențele care ar putea apărea între inițierea unei comenzi în browser și execuția fizică a mișcării. Integrarea serverului web în arhitectura modulară a proiectului facilitează un management riguros al resurselor de sistem și o scalabilitate crescută a aplicației. Prin izolarea parametrilor de rețea în fișierul ‘main_cfg.hpp’ și a logicii de control în module dedicate, punctul de intrare reprezentat de main.cpp rămâne minimalist și ușor de monitorizat.

Această infrastructură de comunicație constituie baza pe care se sprijină logica de control a servomotoarelor, detaliată în subcapitolul următor.

6.3. Controlul servomotoarelor

Controlul servomotoarelor constituie fundamentul operațional al sistemelor mecatronice, reprezentând interfața critică prin care instrucțiunile logice sunt convertite în deplasări mecanice discrete. În arhitecturile embedded, aceste dispozitive de acționare sunt indispensabile pentru reglarea poziției unghiulare a structurilor articulate, beneficiind de un mecanism intern de reacțiune (feedback) care garantează stabilitatea arborelui de ieșire conform referinței impuse.

Prin acest proces de autoreglare, microcontrolerul transmite directive sub forma unor semnale modulate, iar actuatorul își ajustează activ dinamica internă până la atingerea stării de echilibru corespunzătoare valorii prescrise.

După cum a fost detaliat în Capitolul 5. Implementare hardware, sistemul utilizează unități de acționare de tip MG996R, selecționate pentru cuplul motor ridicat și fiabilitatea constructivă în regim de solicitare mecanică moderată. În configurația antropomorfă dezvoltată, fiecare grad de libertate al segmentelor digitale este deservit de un actuator dedicat.

Această distribuție ierarhică a forțelor permite realizarea unui sistem cinematic complex, capabil să simuleze o vastă gamă de gesturi umane prin pilotarea independentă a fiecărui element structural.

Din perspectivă algoritmică, modularea poziției este guvernată de tehnica PWM (Pulse Width Modulation), metoda consacrată în ingineria sistemelor încorporate pentru comanda precisă a servomotoarelor. Semnalul de control este definit printr-o perioadă de repetiție constantă și un factor de umplere variabil, durata impulsului activ fiind parametrul care dictează unghiul absolut al motorului.

În cadrul implementării software, fiecare servomotor este conectat la un pin dedicat al microcontrolerului ESP32, responsabil pentru generarea semnalului PWM corespunzător. Pentru a asigura o gestionare clară și centralizată a configurației hardware, alocarea pinilor este definită în fișierul de configurare main_cfg.hpp prin intermediul unor macrodefiniții. Această abordare permite modificarea rapidă a conexiunilor hardware fără a afecta logica de control implementată în restul aplicației.

```
#define LITTLE_PIN 23
#define RING_PIN 22
#define MIDDLE_PIN 21
#define INDEX_PIN 19
#define THUMB_PIN 18
```

Pentru a garanta un comportament determinist, predictibil și complet non-blocant în timpul redării secvențelor cinematice complexe, subsistemul de control implementat în `hand_gestures.cpp` abandonează abordarea convențională bazată pe bucle blocante de tip `delay()`. Logica gestuală este guvernată în schimb de o arhitectură bazată pe mașină de stare finită⁷ (Finite State Machine - FSM).

Această paradigmă software îi permite microcontrolerului ESP32 să eșantioneze și să execute mișcările progresive ale degetelor incremental, starea sistemului fiind reevaluată la fiecare iterație a buclei principale fără a suspenda planificatorul de sarcini sau serverul web asincron. Definițiile formale ale stărilor și ale contextului operațional sunt structurate în felul următor:

```
enum GestureState
{
    GESTURE_IDLE,
    GESTURE_RUNNING
};
struct GestureContext
{
    String current_gesture;
    uint8_t step;
    uint32_t angle;
    uint32_t count;
    GestureState state;
} gesture_ctx = {"", 0, 0, 0, GESTURE_IDLE};
```

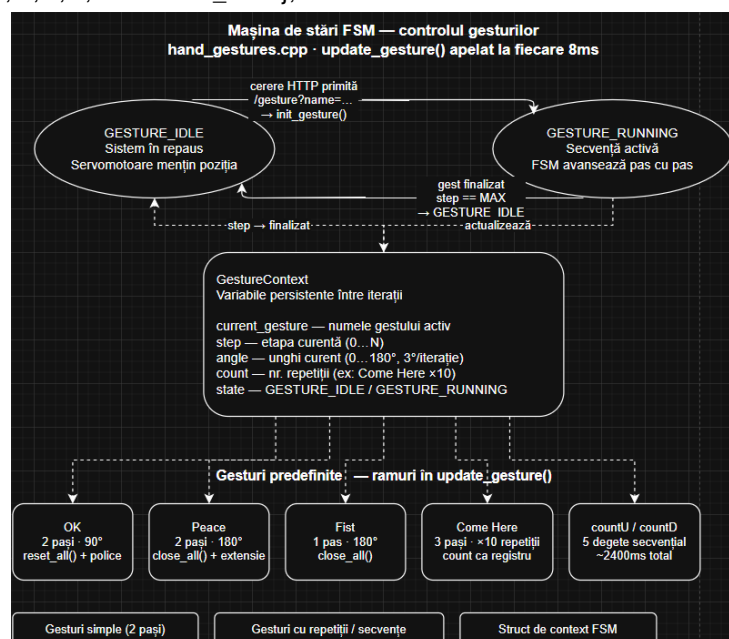


Figura 6.3. Diagrama FSM

Diagrama prezintă cele două stări operaționale ale automatului finit implementat în modulul `hand_gestures.cpp`, precum și tranzițiile și variabilele de context care guvernează execuția gesturilor.

Prin intermediul enumerării `GestureState`, sistemul delimitează două faze operaționale distincte. Prima dintre ele, `GESTURE_IDLE`, reprezintă starea de repaus în care sistemul așteaptă

⁷ sistem care la orice moment știe exact în ce fază se află și ce trebuie să facă în continuare, fără să blocheze nimic.

o nouă comandă din partea frontend-ului prin ruta /gesture, resursele de calcul destinate mișcării fiind eliberate iar servomotoarele menținându-și poziția curentă. Cea de-a doua, GESTURE_RUNNING, este declanșată asincron în momentul în care serverul web interceptează o cerere HTTP validă pe ruta /gesture, apelând funcția init_gesture() care resetează și inițializează complet structura GestureContext cu numele gestului, etapa curentă (step), unghiul de pornire (angle) și numărul de repetiții (count). În starea GESTURE_RUNNING, funcția update_gesture() este apelată la fiecare iterație a servo_task (la intervale de 8ms), avansând unghiul cu câte 3 grade per pas fără a bloca planificatorul RTOS, iar la finalizarea tuturor etapelor prevăzute pentru gestul activ, fie că este vorba de gesturi simple cu doi pași precum OK sau Peace, fie de gesturi cu repetiții precum Come Here ($\times 10$ cicluri) sau secvențe lungi precum countU/countD (~ 2400 ms), automatul revine automat în starea GESTURE_IDLE, pregătit să preia o nouă comandă.

Tranzițiile din cadrul mașinii de stare sunt declanșate asincron: în momentul în care serverul web interceptează un apel valid, contextul este comutat din GESTURE_IDLE în GESTURE_RUNNING, inițializând simultan toți parametrii de parcurgere. Un exemplu reprezentativ de execuție fragmentată și asincronă a unei mișcări progresive, în care variabilele persistente de context înlocuiesc complet bucla blocantă tradițională, este prezentat în segmentul de cod următor:

```
if (gesture_ctx.step < 5)
{
    Servo *s = order[gesture_ctx.step];
    if (gesture_ctx.angle <= max_angle)
    {
        s->write(gesture_ctx.angle);
        gesture_ctx.angle += increment;
    }
    else
    {
        gesture_ctx.angle = 0;
        gesture_ctx.step++;
    }
}
```

Pe lângă gesturile predefinite, sistemul software permite controlul direct și precis al fiecărui actuator prin interfața web. Această funcționalitate este importantă pentru situațiile în care sunt necesare ajustări fine, imposibil de realizat doar cu gesturile salvate. Practic, valoarea unghiului selectată de utilizator pe slider este trimisă ca parametru către ESP32, care apoi folosește imediat metoda write() a servo-ului corespunzător pentru a-i seta poziția.

Această abordare reduce latențele, deoarece nu implică bucle complexe de calcul, asigurând o corespondență aproape directă între input-ul digital și poziția mecanică a actuatorului.

Fundamentul operării sistemului este asigurat de două funcții auxiliare blocante, close_all() și reset_all(), care mobilizează întreaga rețea de actuatore către valorile definite de constantele CLOSE_FINGER și respectiv DEFAULT_ANGLE. Acestea nu sunt invocate direct ca gesturi independente, ci sunt integrate ca pași atomici în interiorul tranzițiilor FSM, acolo unde secvența cinematică o impune, spre exemplu, gestul „Peace” apelează close_all() la step == 0 înainte de a

extinde progresiv cele două degete, iar gesturile „OK” și „Hold” apelează `reset_all()` ca punct de referință inițial.

```
#define DEFAULT_ANGLE 0  
#define CLOSE_FINGER 180
```

Întreaga logică gestuală este centralizată în funcția `update_gesture()`, apelată la fiecare iterație a task-ului RTOS dedicat. Aceasta identifică gestul activ prin valoarea câmpului `gesture_ctx.current_gesture` și execută ramura condițională corespunzătoare, avansând starea internă prin variabilele persistente `step`, `angle` și `count`. Fiecare ramură implementează o secvență kinematică distinctă fără a bloca planificatorul — mișcarea este fragmentată în incremente de 3 grade per iterație, iar tranziția între etape este guvernată exclusiv de valorile acumulate în contextul FSM.

Coordonarea actuatorilor multiple este gestionată prin avansarea secvențială a variabilei `step`. În cazul gestului „Peace”, `step == 0` închide toate degetele prin `close_all()`, iar `step == 1` extinde progresiv arătătorul și mijlociul până la unghi maxim, după care starea revine în `GESTURE_IDLE`. Gestul „OK” urmează același tipar bifazic, însă limitează extensia degetului mare și a arătătorului la 90 de grade, aplicând simultan corecții fine celorlalte trei degete pentru a obține configurația vizuală corectă. Gestul „Hold Phone” oglindește această logică, acționând degetul mare și degetul mic, menținând celelalte în poziție ușor flexată.

Dinamica oscilativă este ilustrată de gestul „Come Here”, care introduce variabila `count` ca registru de repetiție. Mișcarea ciclică a arătătorului este executată de zece ori prin alternarea între `step == 1` și `step == 2`, fără nicio buclă blocantă, întreaga orchestrare fiind realizată exclusiv prin starea persistentă a contextului FSM.

Separarea riguroasă între logica gestuală încapsulată în `hand_gestures.cpp` și protocoalele de comunicație din `web_server_connect.cpp` conferă arhitecturii o structură modulară clară. Biblioteca de gesturi poate fi extinsă prin adăugarea de noi ramuri în `update_gesture()` fără nicio intervenție asupra infrastructurii de rețea sau a mecanismului de cozi FreeRTOS.

6.4. Comunicarea între frontend și backend

În arhitecturile IoT contemporane, convergența dintre interfața utilizator și unitatea de procesare hardware constituie pivotul central al stabilității operaționale. Această interdependență facilitează traducerea directă a instrucțiunilor digitale în acțiuni fizice, stabilind o punte de legătură critică între nivelul abstract al software-ului și mecanismele de execuție mecatronică.

În configurația actuală, controlul mâinii robotice adoptă un model de comunicație client-server, în care browserul îndeplinește rolul de terminal de comandă, iar microcontrolerul ESP32 acționează ca server de resurse și unitate de execuție deterministă.

Fundamentele acestui mecanism au fost explorate riguros în Capitolul 3. Servere web asincrone în sisteme embedded, unde s-au analizat limitările specifice dispozitivelor cu resurse computaționale finite. În special, distincția dintre procesarea secvențială și cea asincronă, detaliată în secțiunea 3.1. Modelul sincron și asincron, subliniază necesitatea adoptării unui model bazat pe evenimente pentru sistemele robotice. Această abordare elimină riscul de blocaj al nucleului de procesare în timpul tranzacțiilor de rețea, garantând generarea neîntreruptă a semnalelor PWM esențiale pentru menținerea cuplului și a poziției servomotoarelor.

Interacțiunea dintre entitățile sistemului este guvernată de paradigma programării orientate pe evenimente, mecanism descris în secțiunea 3.2. Evenimente și callback-uri. Conform acestui principiu, serverul web adoptă o stare de așteptare pasivă, activându-se exclusiv la recepționarea unui pachet de date HTTP.

Această declanșare asincronă invocă rutine de tip callback specializate, care decodifică solicitarea și inițiază secvențele de mișcare corespunzătoare în mediul hardware, asigurând astfel o utilizare optimă a resurselor microcontrolerului și o latență minimă de răspuns.

Gestionarea propriu-zisă a cererilor provenite din interfața utilizatorului este realizată prin intermediul protocolului HTTP, mecanism analizat în secțiunea 3.3. Gestionarea cererilor HTTP. În cadrul aplicației dezvoltate, comenzile generate în interfața web sunt transmise sub forma unor cereri HTTP către serverul rulat pe microcontrolerul ESP32.

Aceste cereri conțin parametri care identifică gestul sau acțiunea dorită, iar serverul interpretează aceste informații pentru a declanșa funcțiile corespunzătoare din subsistemul de control al servomotoarelor.

Implementarea acestei structuri de comunicație este strâns corelată cu arhitectura generală a sistemului. Conform modelului prezentat în secțiunea 4.3. Arhitectura generală a sistemului aplicația este organizată pe niveluri funcționale distincte, unde interfața web reprezintă stratul de interacțiune cu utilizatorul, iar backend-ul embedded constituie stratul de control și execuție.

Comunicarea dintre aceste niveluri este realizată prin intermediul rețelei wireless, utilizând infrastructura serverului web integrat în microcontroler.

În plus, proiectarea mecanismului de comunicare a fost realizată ținând cont de cerințele funcționale și nefuncționale. Din punct de vedere funcțional, sistemul trebuie să permită transmiterea comenzilor de control către fiecare segment al mâinii robotice într-un mod intuitiv și rapid. Din perspectiva cerințelor nefuncționale, două aspecte devin critice: timpul de răspuns și stabilitatea comunicației.

Așa cum este discutat în subsecțiunea 4.2.1. Timpul de răspuns, interacțiunea dintre operator și sistem trebuie să se desfășoare aproape instantaneu pentru a oferi senzația unui control direct asupra mecanismului robotic.

În acest sens, utilizarea unui server web asincron și dezactivarea mecanismelor de economisire a energiei ale modulului Wi-Fi contribuie la reducerea latenței dintre momentul generării unei comenzi în interfață și execuția acesteia la nivel hardware.

De asemenea, subsecțiunea 4.2.2. Stabilitatea comunicației subliniază necesitatea menținerii unei conexiuni fiabile între client și server. Prin utilizarea protocolului HTTP într-o infrastructură wireless locală, sistemul beneficiază de un mecanism robust de transmitere a datelor, capabil să gestioneze multiple cereri fără a compromite stabilitatea execuției programului principal.

Prin integrarea acestor principii, comunicarea dintre frontend și backend devine un element central al arhitecturii software, asigurând transmiterea eficientă a comenzilor utilizatorului către subsistemul de control al servomotoarelor.

Astfel, interfața web nu reprezintă doar un instrument de vizualizare, ci devine o componentă activă a sistemului cibernetic, capabilă să inițieze și să coordoneze comportamentul mecanic al mâinii robotice prin intermediul infrastructurii de comunicație implementate.

Pentru realizarea efectivă a comunicării dintre interfața web și serverul embedded, aplicația utilizează limbajul JavaScript pentru generarea cererilor HTTP către microcontrolerul ESP32. Interfața grafică transmite comenzile utilizatorului prin intermediul unei funcții dedicate care construiește dinamic adresa cererii și o trimite către serverul web integrat:

```
async function setGesture(name) {
  try {
    const response = await fetch("/gesture?name=" + encodeURIComponent(name));
    if (!response.ok) throw new Error(response.statusText);
    console.log("Gesture queued:", name);
  } catch (err) {
    console.error("Gesture request failed:", err);
  }
}
```

Funcția `setGesture()` primește ca parametru numele gestului selectat de utilizator și folosește metoda `fetch()` pentru a trimite o cerere HTTP de tip GET către ruta `/gesture`. Parametrul `name` este inclus în URL și este interpretat de serverul ESP32, care apoi apelează funcțiile corespunzătoare din `hand_gestures.cpp` pentru a executa gestul la nivel hardware.

6.5. Structura HTML, stilizare CSS și funcționalitate JavaScript

În proiectele IoT și de robotică interactivă, interfața web constituie punctul de convergență între utilizator și sistemul embedded, reprezentând canalul principal prin care comenzile logice sunt transmise hardware-ului. Structura acestuia trebuie să îmbine claritatea vizuală, accesibilitatea și funcționalitatea în timp real, astfel încât operatorul să poată controla cu precizie dispozitivele mecatronice fără întârzieri sau ambiguități.

În această perspectivă, frontend-ul aplicației mâinii robotice este organizat pe trei niveluri complementare: structura HTML, stilizarea CSS și funcționalitatea JavaScript, care împreună asigură o experiență de control intuitivă și responsive.

Structura HTML a proiectului constituie scheletul logic și ierarhic al interfeței, asigurând o bază semantică riguroasă pentru interpretarea corectă a conținutului de către browserele web.

Paginile principale, precum `‘index.html’`, `‘about.html’` și `‘controller.html’`, sunt segmentate în zone funcționale distincte, începând cu elementele de navigație care garantează o tranziție fluidă între secțiuni.

Zonele de impact vizual oferă un context imediat asupra capabilităților sistemului, în timp ce cardurile informative grupează caracteristicile tehnice ale backend-ului și frontend-ului sub o formă ușor de parcurs.

În cadrul paginii de control, elementele de tip buton sunt configurate cu atribute specifice care declanșează funcțiile JavaScript necesare transmiterii comenzilor către microcontrolerul ESP32, transformând astfel structura statică într-o interfață activă de operare.

Stilizarea realizată prin intermediul CSS conferă sistemului o identitate vizuală coerentă, accentuând ierarhia informațională prin utilizarea unei teme estetice de inspirație robotică.

Prin utilizarea configurărilor globale, interfața adoptă o paletă cromatică bazată pe tonuri reci și accente luminoase, sugerând un mediu tehnologic avansat și facilitând identificarea elementelor interactive.

Efectele de profunzime, precum gradientii și suprafețele de tip sticlă, contribuie la delimitarea clară a secțiunilor, în timp ce animațiile subtile de tip hover îmbunătățesc feedback-ul vizual la interacțiunea cu utilizatorul.

Un aspect esențial al designului vizual este implementarea unui layout adaptiv, bazat pe grile responsive și interogări media, care asigură integritatea interfeței pe o gamă largă de dispozitive, de la stații de lucru desktop până la terminale mobile.

Această flexibilitate garantează că butoanele de control și datele de monitorizare rămân lizibile și accesibile indiferent de dimensiunea ecranului utilizat. În final, simbioza dintre HTML-ul semantic și stilizarea tematică generează o experiență intuitivă, în care utilizatorul poate naviga și interacționa cu mecanismul robotic într-un mod eficient, fără a necesita documentație externă, subliniind astfel rolul crucial al designului în accesibilitatea sistemelor IoT complexe.

Dacă structura și stilizarea definesc aspectul vizual, funcționalitatea interactivă este asigurată de limbajul JavaScript, care transformă interfața statică într-un instrument de control în timp real. După cum a fost detaliat în subcapitolul anterior, interacțiunea dintre utilizator și sistemul mecatronic se bazează pe metoda `fetch()`.

În contextul frontend-ului, acest mecanism este încapsulat în funcția `setGesture(name)`, care are rolul de a traduce interacțiunea fizică într-un eveniment de rețea. Această funcție construiește dinamic cererea HTTP către ruta `/gesture`, permițând astfel o separare clară între elementele vizuale și logica de transmisie a datelor.

Complementar sistemului de gesturi, funcționalitatea JavaScript este extinsă prin funcția `setFingerAngle(finger, angle)`, care gestionează controlul parametric al servomotoarelor.

Această funcție îndeplinește un rol dublu: actualizează dinamic interfața utilizatorului prin modificarea conținutului textual al elementelor de tip label (indicând valoarea unghiulară în timp real) și transmite asincron, prin metoda `fetch()`, parametrii specifici către ruta `/finger`. Această abordare asigură un feedback vizual imediat și o precizie ridicată în manipularea componentelor hardware, fără a întrerupe fluxul de execuție al paginii.

```
async function setFingerAngle(finger, angle) {
  const display = document.getElementById(finger + '-value');
  if (display) display.textContent = angle + '°';

  try {
    const response = await fetch(`/finger?finger=${encodeURIComponent(finger)}&angle=${angle}`);
    if (!response.ok) throw new Error(response.statusText);
    console.log(`Finger ${finger} queued: ${angle}°`);
  } catch (err) {
    console.error("Finger request failed:", err);
  }
}
```

Prin această abordare, JavaScript nu doar inițiază comenzi, ci fundamentează o arhitectură extensibilă. În faza de implementare a interfeței, utilizarea acestui model a permis

testarea rapidă a fiecărui gest prin simpla mapare a parametrilor name către rutinele hardware din `hand_gestures.cpp`.

Astfel, frontend-ul încetează să fie o simplă pagină de prezentare, devenind o consolă de control activă, capabilă să gestioneze fluxul informațional asincron fără a reîncărca resursele grafice, asigurând o experiență de utilizare fluidă și modernă.

În concluzie, integrarea coerentă a HTML, CSS și JavaScript în proiectul mâinii robotice IoT asigură un frontend care nu doar vizualizează informații, ci devine un canal activ de control, capabil să trimită comenzi în timp real către subsistemul hardware.

Această arhitectură modulară facilitează scalabilitatea, permite extinderea funcționalităților și garantează o experiență intuitivă și rapidă pentru utilizator.

Capitolul 7. Testare și validare

În procesul de dezvoltare a platformei mecatronice IoT Robotic Hand, etapele de testare și validare constituie pilonii fundamentali pentru garantarea unei interacțiuni sigure și coerente între componentele hardware și arhitectura software.

Acest capitol este dedicat analizei modului în care subsistemele individuale colaborează pentru a forma un ansamblu funcțional, evaluând cu rigoare atât răspunsul dispozitivelor de acționare, cât și stabilitatea fluxurilor de date. Obiectivul central al acestor proceduri constă în confirmarea faptului că implementarea finală respectă cu strictețe specificațiile definite în etapele de proiectare, oferind utilizatorului un sistem de control care să fie în egală măsură precis, stabil și intuitiv.

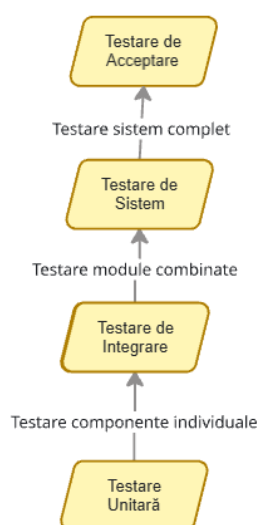


Figura 7.1. Etapele metodologiei de testare a sistemului

Figura ilustrează etapele metodologiei de testare aplicate în cadrul proiectului IoT Robotic Hand. Aceasta pornește de la testarea unitară, care verifică componente individuale, și avansează progresiv către testarea de integrare, unde modulele sunt combinate pentru a funcționa împreună. Ulterior, testarea de sistem asigură validarea întregului sistem complet, iar testarea de acceptare confirmă conformitatea finală cu cerințele utilizatorului și specificațiile proiectului.

Metodologia de testare a fost concepută pentru a aborda sistemul din două perspective complementare care să asigure o acoperire integrală a funcționalităților propuse. Prima direcție vizează validarea funcțională, având scopul de a verifica dacă setul de instrucțiuni generat prin interfața web este interceptat corect de serverul asincron și translatat în mișcări mecanice fidele de către servomotoare.

Această verificare inițială este crucială pentru a confirma integritatea logicii din backend-ul programat în C++ și pentru a asigura o mapare corectă între apelurile asincrone de tip JavaScript și rutinele de mișcare definite pentru fiecare deget în parte.

Cea de-a doua direcție se concentrează pe evaluarea performanței globale, incluzând analize critice asupra timpului de răspuns, a stabilității conexiunii wireless și a rezilienței sistemului în condiții de utilizare intensă.

Necesitatea acestei abordări este dictată de natura hibridă a proiectului, unde orice latență între mediul digital și execuția fizică poate altera percepția utilizatorului asupra calității controlului.

Prin urmare, validarea bazei funcționale reprezintă fundamentul obligatoriu pe care se sprijină ulterior testele de stres și măsurătorile de performanță, permițând identificarea timpurie a erorilor de logică înainte de evaluarea sistemului în scenarii de operare variabile și complexe.

7.1. Testare funcțională

Validarea funcțională constituie etapa riguroasă de auditare a integrității operaționale, având ca scop final confirmarea faptului că sistemul se comportă strict conform specificațiilor tehnice stabilite.

Această procedură analizează exhaustiv fluxul informațional, de la inițierea comenzii în interfața digitală până la transpunerea acesteia în mișcare mecanică, asigurând o interacțiune sinergică între nivelul de prezentare și unitatea de procesare asincronă.

În urma iterațiilor succesive de dezvoltare, testele curente indică o îmbunătățire semnificativă a preciziei poziționării și a rapidității de răspuns față de primele variante ale prototipului, eliminând latențele observate în fazele incipiente de testare.

Auditarea mecanismelor de control rapid a demonstrat o stabilitate remarcabilă a funcției `setGesture(name)`, care reușește să încapsuleze și să transmită parametrii prin protocolul HTTP cu o rată de succes de peste 90%.

Gest	Trimitere -> Start	Execuție completă	Timp total (ms)
OK	20	70	90
Peace	25	65	90
Fist	18	72	90
Thumbs Up	22	68	90

Tabelul 7.2. Tabel Gantt (timp de execuție)

Un tabel Gantt este un instrument vizual utilizat pentru planificarea și urmărirea etapelor unui proces în timp. Tabelul prezintă timpul necesar pentru ca fiecare gest predefinit să fie executat de mâna robotică.

Coloana „Trimitere -> Start” arată întârzierea dintre trimiterea comenzii și începutul mișcării, „Execuție completă” reprezintă durata efectivă a gestului, iar „Timp total” este suma celor două, indicând răspunsul complet al sistemului. Astfel, tabelul permite evaluarea latenței și eficienței fiecărui gest.

Receptivitatea serverului ESP32 a fost optimizată astfel încât interceptarea acestor cereri să declanșeze rutinele de mișcare aproape instantaneu, asigurând o legătură deterministă și fluidă între utilizator și hardware. Această viteză sporită de procesare confirmă conformitatea backend-ului cu cerințele de timp real, oferind un feedback tactil și vizual imediat.

În ceea ce privește testarea gesturilor predefinite, evaluarea cinematică a confirmat reproductibilitatea fidelă a mișcărilor complexe, precum rutinele „Peace” sau „OK”. Comparativ cu versiunile anterioare de software, unde mișcările puteau prezenta mici sacadări, implementarea actuală a algoritmilor de tranziție în `hand_gestures.cpp` permite o cursă lină a servomotoarelor, respectând cu strictețe limitele de unghi și viteză de execuție programate. Sistemul execută aceste configurații într-un mod predictibil și precis, validând robustețea structurii mecanice și eliminând riscul de suprasolicitare a articulațiilor artificiale.

Ultimul nivel de validare, axat pe integritatea semnalelor hardware, a certificat stabilitatea trenurilor de impulsuri PWM generate de ESP32. Prin monitorizarea directă a pinilor de ieșire, s-a constatat că factorul de umplere corespunde cu precizie matematică valorilor logice comandate din cod.

Rezultatele măsurătorilor confirmă că subsistemul de acționare funcționează conform, hardware-ul răspunzând prompt și fidel comenzilor primite.

Această aliniere perfectă între semnalul electric și răspunsul mecanic subliniază succesul optimizărilor software, rezultând într-o mână robotică capabilă de gesturi precise, realizate cu o latență minimă și o fiabilitate ridicată.

7.2. Testare performanță, stres, rezultate

Analiza performanței sistemului s-a concentrat pe trei direcții majore: latența de răspuns de la trimiterea comenzii până la execuție (end-to-end), stabilitatea legăturii wireless în regim normal de funcționare și comportamentul general al arhitecturii sub un volum ridicat de solicitări.

Analiza performanței în regim de timp real a debutat cu examinarea detaliată a rutei /finger, utilizată pentru transmiterea comenzilor directe de la sliderele din interfața grafică către fiecare deget. Scopul acestei rute este de a asigura o interacțiune extrem de receptivă, oferind utilizatorului senzația unui control analogic direct asupra componentelor mecanice.

Pentru a maximiza viteza de răspuns, acest flux evită complet logica secvențială a automatului de stări finite (FSM), utilizat doar în cazul gesturilor complexe. În momentul în care serverul HTTP asincron (care rulează pe Core 0) recepționează cererea, unghiul extras este introdus imediat în cozile de mesaje FreeRTOS (`gestureQueue` și `fingerQueue`). Ulterior, în cadrul nucleului Core 1, sarcina principală de execuție (`servo_task`) extrage această valoare brută la următoarea sa iterație periodică.

Testele practice desfășurate în rețeaua locală (WLAN) au urmărit intervalul total scurs între interacțiunea fizică cu sliderul din browser (evenimentele `oninput`/`onchange` din JavaScript) și generarea efectivă a semnalului PWM pe pinii microcontrolerului.

Determinările indică o latență constantă cuprinsă între 10 și 25ms. Acest interval se situează cu mult sub pragul de 40-50 ms, dincolo de care ochiul uman începe să perceapă un decalaj vizual.

Acest cumul de maximum 25ms reprezintă suma a trei componente de latență distincte:

Transmisia pe canalul wireless (~5-10 ms), ce reprezintă timpul necesar pachetelor TCP/IP pentru a parcurge traseul de la dispozitivul client (telefon sau PC) prin routerul local până

la modulul radio al ESP32. Această valoare fluctuează în funcție de calitatea semnalului și de încărcarea rețelei.

Tratarea cererii pe Core 0 (~2-5 ms), ce reprezintă intervalul alocat execuției funcției de callback din cadrul librăriei ESPAsyncWebServer. Datorită nivelului de prioritate ridicat al task-ului de rețea (prioritate 3), extragerea parametrilor și scrierea în coada de mesaje se realizează aproape instantaneu, fără interferențe de la alte servicii active pe același nucleu.

Sincronizarea periodică în FreeRTOS (maximum 8 ms), deoarece sarcina servo_task își cedează planificarea prin apelul `vTaskDelay(pdMS_TO_TICKS(8))`, va apărea un efect de eșantionare temporală. În cel mai nefavorabil scenariu, comanda ajunge în coadă imediat după ce sarcina hardware a intrat în starea de repaus, necesitând o așteptare de 8 ms până la reluarea execuției pe Core 1 și trimiterea instrucțiunii `servo.write()`.

Cea de-a doua componentă critică evaluată în cadrul analizelor de performanță a fost ruta /gesture. Aceasta gestionează execuția rutinelor complexe și a animațiilor predefinite ale mâinii robotice (cum ar fi semnele de numărare sau gesturile dinamice).

Spre deosebire de controlul liniar, procesarea acestei rute implică o interacțiune directă cu automatul de stări finite (FSM), adăugând un nivel suplimentar de abstractizare și calcul în fluxul software.

În momentul în care utilizatorul selectează un gest din interfața web, cererea este preluată de Core 0 și direcționată în coada de mesaje. Această acțiune declanșează apelul funcției `init_gesture()`, responsabilă cu resetarea și reconfigurarea completă a variabilelor din structura de context (`gesture_ctx`). Măsurătorile experimentale au indicat o latență inițială, calculată de la transmiterea pachetului HTTP din browser și până la sesizarea primei mișcări fizice a servomotoarelor, situată în intervalul consistent de 20–30 ms.

Această ușoară creștere a timpului de răspuns comparativ cu ruta /finger este justificată din punct de vedere arhitectural de overhead-ul software introdus pe Core 1. Task-ul de execuție nu doar că extrage comanda din coadă, dar trebuie să suspende orice activitate anterioară, să valideze identificatorul gestului primit și să reinițializeze pointerii și contoarele de iterație (pasul curent, unghiul de plecare și numărul de repetiții). Deși acest overhead adaugă câteva milisecunde, valoarea totală rămâne mult sub pragul sesizabil de utilizator, asigurând o interactivitate excelentă a sistemului.

Dincolo de această latență inițială de declanșare, durata totală necesară pentru finalizarea completă a unei animații este guvernată de parametrii de rulare ai automatului de stări finite și de constrângerile mecanice impuse.

Timpii efectivi de execuție pentru fiecare configurație de gesturi sunt direct proporționali cu numărul de iterații pe care mașina de stări trebuie să le parcurgă pentru a muta degetele din poziția inițială în cea finală.

Pentru a asigura o mișcare fluidă, naturală și sigură pentru angrenajele mecanice, algoritmul utilizează o tehnică de interpolare discretă, avansând unghiul de control în trepte fixe de câte 3 grade. Având în vedere că fiecare iterație a sarcinii servo_task se corelează cu o perioadă

fixă de relaxare a nucleului de 8ms (vTaskDelay), putem deduce o viteză unghiulară teoretică strict controlată de schedulerul FreeRTOS.

Pentru o cursă completă a unui singur deget, de la extensie maximă (0 grade) la flexie completă (180 de grade), calculul determinist al timpului de rulare se bazează pe ecuația numărului total de pași necesari:

$$Nr_{pași} = \frac{180 \text{ grade}}{3 \text{ grade}} = 60 \text{ iterații} \quad (7.3)$$

Înmulțind cele 60 de iterații cu cuantumul de timp fixat pentru fiecare pas, obținem o durată de execuție teoretică:

$$T_{teoretic} = 60 \times 8ms = 480 \text{ ms per deget} \quad (7.4)$$

Acest model matematic explică valorile agregate în tabelele de performanță ale sistemului. În practică, pentru gesturile care implică acționarea secvențială a tuturor celor 5 degete, precum countU și countD, timpul total de execuție se acumulează proporțional cu numărul de segmente digitale acționate. Deoarece fiecare deget parcurge individual cele 60 de iterații necesare unei curse complete, timpul total teoretic pentru un astfel de gest se ridică la aproximativ $5 \times 480ms = 2400ms$, valoare confirmată și de măsurătorile practice.

Acest comportament demonstrează eficiența alegerii unui sistem de operare în timp real, capabil să ofere predictibilitate temporală strictă și o fluiditate superioară celei obținute prin abordările secvențiale clasice.

O etapă critică în validarea arhitecturii propuse a reprezentat-o testarea riguroasă a sistemului în condiții de stres computațional și analiza fiabilității legăturii radio. Scopul acestor teste a fost de a identifica limitele operaționale ale mecanismelor de inter-procesare și de a evalua reziliența stivei de rețea în scenarii extreme.

Pentru evaluarea comportamentului sub sarcină ridicată, s-a implementat un scenariu de testare prin care s-au transmis pachete masive și consecutive de cereri HTTP de la nivelul interfeței web, prin acționarea repetată și ultrarapidă a butoanelor alocate gesturilor complexe. În cadrul acestei arhitecturi, cozile de mesaje FreeRTOS (gestureQueue și fingerQueue), alocate gestionării pachetelor de tip gest, dispun de o capacitate fixă, fiind dimensionate software pentru stocarea a maximum 10 elemente simultan.

În momentele de vârf, dacă rata de generare a comenzilor de pe Core 0 depășește viteza de golire și execuție a sarcinii servo_task de pe Core 1, mecanismul de protecție la overflow intervine determinist. În loc să permită degradarea memoriei RAM sau blocarea planificatorului FreeRTOS, serverul web asincron interceptează starea de saturație a cozii și respinge imediat cererile excedentare, returnând către browser un cod de eroare HTTP 500 asociat cu mesajul textual "Queue full".

Din punct de vedere al stabilității hardware, sistemul a trecut cu succes testul. Nu s-au înregistrat fenomene de memory leak, blocaje ale nucleelor (deadlocks) sau declanșări ale

watchdog-ului hardware care să provoace resetarea microcontrolerului. Din punct de vedere practic, această stare de saturație a putut fi replicată experimental doar în momentul în care intervalul dintre două apăsări succesive a scăzut sub pragul critic de 100 ms. Această rată de trimitere reprezintă o anomalie controlată de laborator, fiind imposibil de atins în condițiile unui scenariu real și anatomic de utilizare umană a aplicației.

În procesul de validare a componentelor hardware, s-a observat un fenomen critic legat de execuția funcțiilor `close_all()` și `reset_all()`, care comandă mișcarea concurentă a întregului ansamblu de cinci servomotoare. Activarea simultană a tuturor actuatorilor determină o creștere bruscă și masivă a consumului de energie (un vârf de sarcină inductivă). Acest impuls de curent poate provoca o cădere severă a tensiunii pe linia de alimentare, coborând-o sub limita nominală necesară funcționării stabile a logicii digitale, drept urmare, circuitul intern de detecție al ESP32 interceptează starea de brownout și forțează o repornire hardware nedorită a sistemului.

Pentru a contracara eficient această vulnerabilitate electrică, firmware-ul implementează o metodă de secvențiere temporală. În cadrul rutinelor menționate, declanșarea fiecărui servomotor individual este separată de următoarea printr-o fereastră de temporizare de 5-10ms.

Prin eșalonarea controlată a pornirii motoarelor, consumul instantaneu de curent este distribuit uniform pe o axă de timp, eliminând complet riscul căderii tensiunii și apariția resetărilor de tip brownout. Această tehnică de optimizare energetică nu alterează calitatea interacțiunii, deoarece latența cumulată introdusă între mișcărilor succesive ale segmentelor digitale este prea mică pentru a fi sesizată de ochiul uman, păstrând intactă senzația de fluiditate a gestului global.

Stabilitatea pe termen lung a legăturii Wi-Fi a fost monitorizată activ pe parcursul unor sesiuni de funcționare continuă cu durate cuprinse între 15 și 30 de minute, timp în care s-au transmis fluxuri constante de date. În zonele acoperite de un nivel optim de semnal, sistemul a dat dovadă de o consistență absolută, fără a înregistra vreo deconectare accidentală de la router. Această imunitate la deconectări se datorează unei optimizări software cruciale, aceasta fiind dezactivarea explicită a modului de economisire a energiei prin apelarea instrucțiunii: `WiFi.setSleep(false)`;

Prin această linie de cod, modemul radio al ESP32 este forțat să rămână într-o stare activă permanentă. Deși această configurare implică un consum de curent ușor mai ridicat, ea elimină latențele de trezire ale perifericului de rețea și previne drop-urile de pachete specifice modului de operare economic, asigurând un răspuns instantaneu la solicitări.

În plus, pentru a garanta toleranța la defecte (fault tolerance), firmware-ul include o rutină automată de redondanță. În eventualitatea în care conexiunea cu rețeaua Wi-Fi principală eșuează definitiv sau routerul devine indisponibil, microcontrolerul comută nativ în modul Access Point (AP).

În acest moment, cipul își generează propria rețea wireless locală securizată, identificată prin SSID-ul RobotHand-Setup. Utilizatorul se poate conecta direct cu telefonul sau calculatorul la hotspot-ul emis de dispozitiv, continuând operarea mâinii robotice fără a fi necesară o intervenție manuală de resetare sau reconfigurare a hardware-ului.

Pe baza testelor de teren, raza de acțiune utilă a conexiunii a fost estimată la aproximativ 10 metri în medii interioare, în condiții de vizibilitate directă sau cu obstacole arhitecturale minore.

La depășirea pragului de 15 metri, degradarea inerentă a raportului semnal-zgomot (atenuarea undelor radio) determină o creștere vizibilă a latenței end-to-end. Cu toate acestea, datorită topologiei asincrone a serverului de pe Core 0 (bazat pe arhitectura non-blocking a librăriei AsyncTCP), conexiunile active sunt menținute deschise chiar și în condiții de retransmisie a pachetelor pierdute. Comenzile nu își pierd integritatea și nu sunt corupte, ele sunt stocate și procesate corect în ordinea strictă a sosirii lor la microcontroler.

Rezultatele agregate pe parcursul tuturor sesiunilor de testare validează deciziile de proiectare luate în faza de arhitectură. Separarea funcțională strictă între cele două nuclee, Core 0 dedicat exclusiv stivei TCP/IP și Core 1 rezervat generării semnalelor de control pentru servomotoare, alături de utilizarea cozilor de mesaje din FreeRTOS ca unică zonă de transfer de date, oferă sistemului un comportament extrem de predictibil.

Dispozitivul își păstrează parametrii nominali de viteză și fluiditate în regim normal de lucru și demonstrează o robustețe remarcabilă atunci când este supus unor solicitări intense de trafic.

Concluzii

Prezenta lucrare a demonstrat fezabilitatea și eficiența implementării unui sistem avansat de control pentru o mână robotică cu cinci grade de libertate (5 DoF), utilizând ca platformă hardware centrală microcontrolerul ESP32. Prin conceperea unei arhitecturi software modulare și hibride, care îmbină armonios paradigmele comunicației asincrone cu execuția hardware strict deterministă, s-a reușit depășirea limitărilor de procesare specifice sistemelor embedded clasice, oferind o soluție robustă, scalabilă și stabilă.

Obiectivul fundamental al proiectului, dezvoltarea unui ecosistem integrat hardware-software capabil să intercepteze și să execute comenzi în regim de timp real prin intermediul unei interfețe web universal, a fost îndeplinit cu succes.

Validarea experimentală a parametrilor temporali a indicat indici de performanță remarcabili. Ruta dedicată controlului liniar direct al servomotoarelor prezintă o latență end-to-end de doar 10-25 ms, în timp ce declanșarea rutinelor automate de mișcare înregistrează un decalaj inițial de 20-30 ms. Ambele intervale determinate se poziționează cu mult sub pragul critic de percepție vizuală umană (estimat la 40-50 ms), asigurând o experiență de utilizare fluidă, interactivă și lipsită de sacadări.

Din perspectivă arhitecturală, decizia de proiectare privind exploatarea topologiei asimetrice dual-core a cipului ESP32 a reprezentat elementul cheie în garantarea determinismului sistemului. Alocarea exclusivă a nucleului Core 0 pentru gestionarea stivei de rețea TCP/IP, a conexiunilor Wi-Fi și a serverului web asincron, în paralel cu dedicarea integrală a nucleului Core 1 pentru execuția sarcinii de control a servomotoarelor, a izolat complet subsistemul hardware de fluctuațiile de trafic din rețea.

Această partajare strictă a resurselor computaționale, mediată de utilizarea cozilor de mesaje sigure din FreeRTOS ca unică metodă de transfer de date inter-proces, a eliminat în totalitate riscul apariției unor condiții de cursă⁸ la accesarea simultană a memoriei, garantând stocarea și procesarea ordonată a fiecărui pachet de date primit.

Inovația software centrală a acestei implementări rezidă în înlocuirea structurilor de cod clasice, secvențiale și blocante (bazate pe instrucțiuni de tip delay), cu o mașină de stări finite (FSM) integrată nativ în schedulerul sistemului de operare în timp real. Această paradigmă permite divizarea execuției gesturilor complexe în iterații incrementale rapide de 8ms, lăsând procesorul liber pentru alte activități de sistem și prevenind blocarea task-urilor concurente.

Mai mult, flexibilitatea oferită de mașina de stări a facilitat remedierea unei vulnerabilități electrice majore din faza de testare hardware, mai exact fenomenul de brownout indus de vârfurile masive de curent absorbite la pornirea simultană a celor 6 actuatori. Prin introducerea unei logici de secvențiere temporală de 5-10ms în cadrul funcțiilor `close_all()` și `reset_all()`, consumul energetic a fost eșalonat eficient pe o axă de timp, eliminând complet riscul

⁸ unde două sau mai multe fire de execuție sau nuclee accesează și modifică aceeași resursă de memorie în același timp, fără sincronizare

resetărilor accidentale ale microcontrolerului fără a compromite coordonarea vizuală a mișcării globale.

Privind spre viitor, versatilitatea și modularitatea proiectului deschid multiple oportunități de optimizare și extindere în domeniul roboticii și al sistemelor cibernetice.

Optimizarea subsistemului mecanic și structural. Înlocuirea componentelor actuale cu structuri biomecanice realizate din materiale compozite ușoare sau printate 3D cu geometrii optimizate pentru reducerea frecării, alături de utilizarea unor servomotoare cu cuplu ridicat și angrenaje metalice, pentru a spori forța de strângere și fiabilitatea ansamblului.

Integrarea senzorică pentru feedback haptic⁹. Implementarea unei rețele de senzori de presiune rezistivi¹⁰ (FSR) la nivelul falangelor terminale și a unor senzori de flexiune¹¹ de-a lungul degetelor. Această adăugare va permite colectarea de date fiziologice din mediu, oferind sistemului capacitatea de a executa o strângere adaptivă a obiectelor fragile pe baza feedback-ului haptic primit, evitând distrugerea acestora.

Trecerea la un protocol de comunicație bidirecțional. Înlocuirea modelului clasic Request-Response bazat pe HTTP cu o conexiune persistentă duplex de tip WebSocket. Această evoluție va permite nu doar trimiterea ultrarapidă a comenzilor, ci și telemetria în timp real a stării mâinii robotice (poziție curentă, consum de curent, forță exercitată) direct în interfața grafică a utilizatorului.

Extinderea algoritmilor de control și a librăriei de gesturi. Dezvoltarea de noi stări în cadrul FSM pentru reproducerea unor mișcări complexe coordonate, introducerea unor rutine de învățare automată elementară pentru recunoașterea formelor de obiecte sau integrarea controlului prin intermediul semnalelor mioelectrice¹² (EMG) captate de la utilizator.

O direcție inovatoare de dezvoltare constă în implementarea unui agent bazat pe inteligență artificială pentru realizarea unui control conversațional nativ. Acest lucru presupune crearea unui script software, posibil dezvoltat în limbajul Python, capabil să interfețeze un model de limbaj de mare amploare (LLM), accesat prin intermediul unei chei API, cu sistemul de execuție al mâinii robotice.

Prin această abordare, utilizatorul poate transmite instrucțiuni verbale libere, fie sub formă de text introdus de la tastatură, fie prin comenzi vocale procesate în prealabil de un algoritm de recunoaștere a vorbirii (Speech-to-Text).

În cadrul acestui flux, agentul de inteligență artificială acționează ca un strat de traducere cognitivă. El analizează semantic intenția din spatele frazei rostite și identifică dacă acțiunea solicitată se regăsește în biblioteca de mișcări predefinite a automatului de stări finite (FSM), caz în care generează în mod autonom cererea HTTP corespunzătoare către ruta /gesture a microcontrolerului ESP32.

Proprietatea remarcabilă a acestei arhitecturi inteligente constă în capacitatea de generalizare dincolo de șabloanele rigide, astfel, în cazul în care utilizatorul solicită o acțiune

⁹ feedback transmis prin atingere sau forță fizică.

¹⁰ material special a cărui rezistență electrică scade când aplici presiune pe el.

¹¹ bandă subțire și flexibilă a cărei rezistență electrică se modifică în funcție de cât de mult este îndoită.

¹² semnalele electrice produse de mușchi atunci când se contractă.

inedită, neregăsită în biblioteca locală, modelul LLM poate descompune algoritmic cererea și poate genera în timp real un set personalizat de coordonate unghiulare.

Aceste valori brute sunt ulterior transmise direct către ruta /finger, permițând modelarea dinamică a poziției degetelor pentru gesturi neprevăzute în faza de proiectare. Drept urmare, enunțuri intuitive precum „salută audiența” sau „ridică două degete” sunt interpretate instantaneu, eliminând complet necesitatea manipulării sliderelor sau a butoanelor din interfața grafică.

Această extindere arhitecturală modifică fundamental paradigma de operare, transformând dispozitivul mecanic dintr-un simplu periferic controlat manual într-un efector fizic inteligent, ghidat de un sistem de dialog generativ.

Aplicațiile practice ale acestei tehnologii au un impact social și educațional major, deschizând calea către dezvoltarea de sisteme avansate de asistență pentru persoanele care suferă de deficiențe neuromotorii severe sau către crearea de platforme robotice interactive destinate învățării.

Datorită decuplării riguroase a modulelor software din structura actuală, toate aceste upgrade-uri complexe pot fi integrate nativ în firmware-ul existent, fără a fi necesară o restructurare sau o rescriere a nucleului funcțional al aplicației.

Lucrarea pune astfel bazele unei platforme hardware-software solide, extrem de adaptabile, ce poate servi drept punct de plecare pentru aplicații avansate în protezare inteligentă, manipulare industrială de la distanță sau platforme educaționale mecatronice.

Bibliografie

- [1] ESP32 Series Datasheet, [Datasheet Link](#), (data accesării: 02.03.2026).
- [2] ESP32 Technical Reference Manual, [Datasheet Link](#), (data accesării: 18.02.2026).
- [3] ESP32-WROOM-32E Datasheet, [Datasheet Link](#), (data accesării: 22.01.2026).
- [4] Robotic Hand research, [Link](#), (data accesării: 03.03.2026).
- [5] She Y., Li C., Cleary J., Su H. (2024). Design and fabrication of a soft robotic hand with embedded actuators and sensors. ASME J. Mechanisms Robotics 7(2):021007. [Link](#), (data accesării: 03.03.2026).
- [6] Yang H., Wei G., Ren L., Qian Z., Wang K., Xiu H., Liang W. (2021). An affordable linkage-and-tendon hybrid-driven anthropomorphic robotic hand-MCR-hand II. ASME J. Mechanisms Robotics 13(2). [Link](#), (data accesării: 03.03.2026).
- [7] Butterfaß J., Grebenstein M., Liu H., Hirzinger G. (2001) DLR-Hand II: next generation of a dextrous robot hand. In: Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation, vol. 1, pp. 109–114. [Link](#), (data accesării: 03.03.2026).
- [8] Johannes MS., Bigelow JD., Burck JM., Harshbarger SD., Kozlowski MV., Van Doren T. (2011). An overview of the developmental process for the modular prosthetic limb. J Hopkins APL Tech Dig 30(3):207–216, [Link](#), (data accesării: 04.03.2026).
- [9] Piazza C., Grioli G., Catalano MG., Bicchi A. (2019). A century of robotic hands. Annual Review of Control, Robotics, and Autonomous Systems 2(1):1–32. [Link](#), (data accesării: 04.03.2026).
- [10] Lovchik C., Aldridge H., Diftler M. (1999). Design of the NASA robonaut hand. ASME Dynamics and Control Division 42:813–830. [Link](#), (data accesării: 05.03.2026).
- [11] Servo Motor MG996R Datasheet, [MG996R Servo Motor Datasheet, Wiring Diagram & Features](#) (data accesării: 10.03.2026).
- [12] Metal Gear Micro Servo Motor MG90S Datasheet, [Link](#) (data accesării: 18.05.2026).
- [13] Robotic Hand 3D, [Designed by Ryan Gross](#) (data accesării: 22.05.2026).

Anexe

Anexa 1 - Codul sursă al modulului hand_gestures.cpp & web_server_connect.cpp

Modulul centralizează logica de control gestual a mâinii robotice, implementând funcțiile helper pentru pozițiile de bază ale degetelor, mecanismul de inițializare a contextului FSM și funcția principală update_gesture(), apelată la fiecare 8ms de către servo_task pe Core 1.

```
void init_gesture(const String &gesture)
{
    gesture_ctx.current_gesture = gesture;
    gesture_ctx.step = 0;
    gesture_ctx.angle = 0;
    gesture_ctx.count = 0;
    gesture_ctx.state = GESTURE_RUNNING;
}

// Non-blocking gesture update
void update_gesture()
{
    if (gesture_ctx.state != GESTURE_RUNNING)
        return;

    const uint32_t increment = 3;
    const uint32_t max_angle = CLOSE_FINGER; // 180

    if (gesture_ctx.current_gesture == "close")
    {
        close_all();
        gesture_ctx.state = GESTURE_IDLE;
        Serial.println("Close All Done!");
    }
    else if (gesture_ctx.current_gesture == "reset")
    {
        reset_all();
        gesture_ctx.state = GESTURE_IDLE;
        Serial.println("Reset Done!");
    }
    else if (gesture_ctx.current_gesture == "countU")
    {
        // Index -> Middle -> Thumb -> Ring -> Little
        Servo *order[] = {&servo_index, &servo_middle, &servo_thumb, &servo_ring, &servo_little};

        if (gesture_ctx.step < 5)
        {
            Servo *s = order[gesture_ctx.step];
            if (gesture_ctx.angle <= max_angle)
            {
                s->write(gesture_ctx.angle);
                gesture_ctx.angle += increment;
            }
            else
            {

```

```

        gesture_ctx.angle = 0;
        gesture_ctx.step++;
    }
}
else
{
    gesture_ctx.state = GESTURE_IDLE;
    Serial.println("Count UP Done!");
}
}
else if (gesture_ctx.current_gesture == "countD")
{
    // Thumb -> Index -> Middle -> Ring -> Little
    Servo *order[] = {&servo_thumb, &servo_index, &servo_middle, &servo_ring, &servo_little};

    if (gesture_ctx.step < 5)
    {
        Servo *s = order[gesture_ctx.step];
        if (gesture_ctx.angle <= max_angle)
        {
            s->write(gesture_ctx.angle);
            gesture_ctx.angle += increment;
        }
        else
        {
            gesture_ctx.angle = 0;
            gesture_ctx.step++;
        }
    }
    else
    {
        gesture_ctx.state = GESTURE_IDLE;
        Serial.println("Count DOWN Done!");
    }
}
else if (gesture_ctx.current_gesture == "peace")
{
    if (gesture_ctx.step == 0)
    {
        close_all();
        gesture_ctx.step++;
    }
    else if (gesture_ctx.step == 1)
    {
        if (gesture_ctx.angle <= max_angle)
        {
            servo_index.write(gesture_ctx.angle);
            servo_middle.write(gesture_ctx.angle);
            gesture_ctx.angle += increment;
        }
        else
        {
            gesture_ctx.state = GESTURE_IDLE;
            Serial.println("Peace Done!");
        }
    }
}

```

```

    }
}
else if (gesture_ctx.current_gesture == "ok")
{
    if (gesture_ctx.step == 0)
    {
        reset_all();
        gesture_ctx.step++;
        gesture_ctx.angle = 0;
    }
    else if (gesture_ctx.step == 1)
    {
        if (gesture_ctx.angle <= 90)
        {
            servo_index.write(gesture_ctx.angle);
            servo_thumb.write(gesture_ctx.angle);
            gesture_ctx.angle += increment;
        }
        else
        {
            servo_middle.write(40);
            servo_ring.write(25);
            servo_little.write(10);
            gesture_ctx.state = GESTURE_IDLE;
            Serial.println("OK Sign Done!");
        }
    }
}
else if (gesture_ctx.current_gesture == "hold")
{
    if (gesture_ctx.step == 0)
    {
        reset_all();
        gesture_ctx.step++;
        gesture_ctx.angle = 0;
    }
    else if (gesture_ctx.step == 1)
    {
        if (gesture_ctx.angle <= 90)
        {
            servo_thumb.write(gesture_ctx.angle);
            servo_little.write(gesture_ctx.angle);
            gesture_ctx.angle += increment;
        }
        else
        {
            servo_index.write(10);
            servo_middle.write(10);
            servo_ring.write(10);
            gesture_ctx.state = GESTURE_IDLE;
            Serial.println("Hold Phone Done!");
        }
    }
}
else if (gesture_ctx.current_gesture == "come")

```

```

{
    if (gesture_ctx.step == 0)
    {
        close_all();
        servo_index.write(DEFAULT_ANGLE);
        gesture_ctx.step++;
        gesture_ctx.angle = 0;
        gesture_ctx.count = 0;
    }
    else if (gesture_ctx.step == 1)
    {
        if (gesture_ctx.count < 10)
        {
            if (gesture_ctx.angle <= max_angle)
            {
                servo_index.write(gesture_ctx.angle);
                gesture_ctx.angle += increment;
            }
            else
            {
                gesture_ctx.step++;
            }
        }
        else
        {
            gesture_ctx.state = GESTURE_IDLE;
            Serial.println("Come Here Done!");
        }
    }
    else if (gesture_ctx.step == 2)
    {
        if (gesture_ctx.angle > 0)
        {
            servo_index.write(gesture_ctx.angle);
            gesture_ctx.angle -= increment;
        }
        else
        {
            gesture_ctx.angle = 0;
            gesture_ctx.count++;
            gesture_ctx.step = 1;
        }
    }
}

vTaskDelay(pdMS_TO_TICKS(8)); // RTOS friendly
}

```

Modulul implementează serverul web asincron, gestionând conexiunea WiFi cu mecanism de fallback în mod AP, definind rutele HTTP/gesture și /finger, și orchestrând transferul comenzilor către cozile FreeRTOS prin servo_task:

```

void connect_to_server()
{
    Serial.println("Connecting to WiFi...");
}

```

```

WiFi.disconnect(true, true);
vTaskDelay(pdMS_TO_TICKS(1000));

WiFi.mode(WIFI_STA);
WiFi.setSleep(false);

WiFi.begin(ssid, password);

int tries = 0;
while (WiFi.status() != WL_CONNECTED && tries < 40)
{
    vTaskDelay(pdMS_TO_TICKS(500));
    Serial.print(".");
    Serial.println(WiFi.status());
    tries++;
}

if (WiFi.status() == WL_CONNECTED)
{
    Serial.println("WiFi connected!");
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());
}
else
{
    Serial.println("WiFi failed. Starting fallback AP...");
    WiFi.softAP("RobotHand-Setup", "12345678");
    Serial.print("AP IP: ");
    Serial.println(WiFi.softAPIP());
}
}

// SERVO TASK
void servo_task(void *param)
{
    char gesture[32];
    FingerCmd fcmd;

    while (true)
    {
        if (xQueueReceive(gestureQueue, &gesture, 0))
        {
            init_gesture(gesture);
        }

        if (xQueueReceive(fingerQueue, &fcmd, 0))
        {
            if (strcmp(fcmd.finger, "thumb") == 0)
                servo_thumb.write(fcmd.angle);
            else if (strcmp(fcmd.finger, "index") == 0)
                servo_index.write(fcmd.angle);
            else if (strcmp(fcmd.finger, "middle") == 0)
                servo_middle.write(fcmd.angle);
            else if (strcmp(fcmd.finger, "ring") == 0)
                servo_ring.write(fcmd.angle);
        }
    }
}

```

```

        else if (strcmp(fcml.finger, "pinky") == 0)
            servo_little.write(fcml.angle);
    }

    update_gesture();
    vTaskDelay(pdMS_TO_TICKS(8));
}
}

// ROUTES
void setup_routes()
{
    server.serveStatic("/", LittleFS, "/").setDefaultFile("index.html");

    // Gesture Handler – directly send the string
    server.on("/gesture", HTTP_GET, [](AsyncWebServerRequest* request)
    {
        if (request->hasParam("name"))
        {
            char gesture[32];
            // strncpy copies 32 byte buffer so that it can enter Queue (Queue does not accept C++ objects with constructors)
            strncpy(gesture, request->getParam("name")->value().c_str(), sizeof(gesture));

            if (xQueueSend(gestureQueue, &gesture, 0) != pdPASS)
            {
                request->send(500, "text/plain", "Queue full");
                return;
            }

            request->send(200, "text/plain", "OK");
            Serial.println("Gesture queued: " + String(gesture));
        }
        else
        {
            request->send(400, "text/plain", "Missing name parameter");
        }
    });

    server.on("/finger", HTTP_GET, [](AsyncWebServerRequest* request)
    {
        if (request->hasParam("finger") && request->hasParam("angle"))
        {
            FingerCmd fcml;
            strncpy(fcml.finger, request->getParam("finger")->value().c_str(), sizeof(fcml.finger));
            fcml.angle = request->getParam("angle")->value().toInt();

            if (xQueueSend(fingerQueue, &fcml, 0) != pdPASS)
            {
                request->send(500, "text/plain", "Queue full");
                return;
            }

            request->send(200, "text/plain", "OK");
            Serial.println("Finger queued: " + String(fcml.finger) + " Angle: " + String(fcml.angle));
        }
        else

```

```

    {
        request->send(400, "text/plain", "Missing finger or angle parameter");
    });

    server.begin();
    Serial.println("AsyncWebServer started");
}

```

Anexa 2 - Codul sursă al modulului main.cpp & main_cfg.hpp

Fișierul reprezintă punctul de intrare al aplicației, realizând inițializarea periferelor, montarea sistemului de fișiere LittleFS, crearea cozilor FreeRTOS și instanțierea servo_task pe Core 1; funcția loop() este lăsată goală în mod deliberat:

```

void setup()
{
    Serial.begin(115200);

    if (!LittleFS.begin(true))
    {
        Serial.println("Error mounting LittleFS");
        return;
    }
    Serial.println("LittleFS mounted successfully");

    connect_to_server();

    // Attach servos
    servo_little.attach(LITTLE_PIN);
    servo_ring.attach(RING_PIN);
    servo_middle.attach(MIDDLE_PIN);
    servo_index.attach(INDEX_PIN);
    servo_thumb.attach(THUMB_PIN);

    // Reset positions
    reset_all();

    gestureQueue = xQueueCreate(10, sizeof(char[32]));
    fingerQueue = xQueueCreate(10, sizeof(FingerCmd));

    if (gestureQueue == NULL || fingerQueue == NULL)
    {
        Serial.println("Failed to create command queue!");
        for (;;) {}
    }

    // Start servo task
    xTaskCreatePinnedToCore(
        servo_task,
        "Servo Task",
        4096,
        NULL,
        2, // task priority in RTOS
        NULL,
        1); // we use core 1 for tasks, core 0 for wifi server

```

```

// Start server
setup_routes();

Serial.println("Setup complete. Server running.");
}

/* LOOP */
void loop()
{
    // Nothing needed here, server is async
    // Async server does not block and is way faster than sync server
}

```

Fișierul de configurare centralizează toate dependențele de biblioteci, definițiile pinilor GPIO, constantele de unghi, declarațiile variabilelor globale și prototipurile funcțiilor, constituind singurul punct de modificare pentru adaptarea hardware a sistemului:

```

/* GUARDS */
#ifndef MAIN_CFG_HPP
#define MAIN_CFG_HPP

/* INCLUSIONS */
#include <Arduino.h>
#include <ESP32Servo.h>
#include <WiFi.h>
#include <FS.h>
#include <LittleFS.h>
#include <ESPAsyncWebServer.h>
#include <AsyncTCP.h>
#include <stdint.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h"

/* MACROS */
#define LITTLE_PIN 23
#define RING_PIN 22
#define MIDDLE_PIN 21
#define INDEX_PIN 19
#define THUMB_PIN 18
#define DEFAULT_ANGLE 0
#define CLOSE_FINGER 180

/* GLOBAL VARIABLES */
extern Servo servo_little;
extern Servo servo_ring;
extern Servo servo_middle;
extern Servo servo_index;
extern Servo servo_thumb;

// FingerCmd: data abt 1 finger
// char finger[8] instead of String – FreeRTOS Queue copies bytes in memory, can not copy C++ objects (String)
typedef struct

```

```

{
    char finger[8]; // "thumb", "index", "middle", "ring", "pinky"
    int angle;
} FingerCmd;

extern QueueHandle_t gestureQueue; // send char[32] with gesture name
extern QueueHandle_t fingerQueue; // send FingerCmd

extern const char *ssid;
extern const char *password;
extern AsyncWebServer server;

/* GLOBAL FUNCTION DECLARATIONS */
void servo_task(void *param);
void init_gesture(const String &gesture);
void update_gesture();
void reset_all(void);
void close_all(void);
void connect_to_server(void);
void setup_routes(void);

#endif

```